Handling Concurrency in Behavior Trees

Michele Colledanchise, *Member, IEEE*, and Lorenzo Natale, *Senior Member, IEEE*

***Abstract*—This article addresses the concurrency issues affecting behavior trees (BTs), a popular tool to model the behaviors of autonomous agents in the video game and the robotics industry. BT designers can easily build complex behaviors composing simpler ones, which represents a key advantage of BTs. The parallel composition of BTs expresses a way to combine concurrent behaviors that has high potential, since composing pre-existing BTs in parallel results easier than composing in parallel classical control architectures, as finite state machines or teleo-reactive programs. However, BT designers rarely use such composition due to the underlying concurrency problems similar to the ones faced in concurrent programming. As a result, the parallel composition, despite its potential, finds application only in the composition of simple behaviors or where the designer can guarantee the absence of conflicts by design. In this article, we define two new BT nodes to tackle the concurrency problems in BTs and we show how to exploit them to create predictable behaviors. In addition, we introduce measures to assess execution performance and show how different design choices affect them. We validate our approach in both simulations and the real world. Simulated experiments provide statistically significant data, whereas real-world experiments show the applicability of our method on real robots. We provided an open-source implementation of the novel BT formulation and published all the source code to reproduce the numerical examples and experiments.**

***Index Terms*—Autonomous systems, behavior trees, behavior-based systems.**

I. Introduction

We study concurrent robot behaviors encoded as behavior trees (BTs) [1]. Robotics applications of BTs span from manipulation [2]–[4] to nonexpert programming [5]–[7]. Other works include task planning [8], human–robot interaction [9]–[11], learning [12]–[15], UAV [16]–[21], multirobot systems [22]–[25], and system analysis [26]–[28]. The Boston Dynamics's Spot uses BTs to model the robot's mission [29], the Navigation Stack and the task planner of ROS2 use BTs to encode the high level robot's behavior [30], [31].

The particular syntax and semantic of BTs, which we will describe in the article, allows creating easily complex behaviors composing simpler ones. A BT designer can compose behaviors in different ways, each with its own semantic. The *Parallel* composition allows a designer to describe the concurrent execution of several subbehaviors. In BTs, this composition scales better as the complexity of the behavior increases, compared to other control architectures where the system's complexity results from the product of its subsystems' complexities [1]. However, the Parallel composition still entails concurrency issues (e.g., race conditions, starvation, deadlocks, etc.), like any other control architecture. As a result, such composition gets applied only to orthogonal tasks.

In the BT literature, the Parallel composition finds applications only to the composition of orthogonal tasks, where the designer guarantees the absence of conflicts. In this article, we show how we can extend the use of BTs to address the concurrency issues above. In particular, we show how to obtain synchronized concurrent BTs execution, exclusive access to resources, and predictable execution times. We define performance measures and analyze how they are affected by different design choices. We also provide reproducible experimental validation by publishing the implementation of our BT library, code, and data related to our experiments.

Software developers from the video game industry conceived BTs to model the behaviors of nonplayer characters (NPCs) [32], [33]. Controlled hybrid systems (CHSs) [34], which combine continuous and discrete dynamics, were a natural formulation of NPCs' execution and control. However, it turned out that CHSs were not fit to program an NPC, as CHSs implement the discrete dynamics in the form of finite state machines (FSMs). The developers realized that FSMs scale poorly, hamper modification and reuse, and have proved inadequate to represent complex deliberation [35], [36]. In this context, the issues lie in the transitions of FSMs, which implement a *one-way control transfer*, similar to a GOTO statement of computer programming. In 1968, Edsger Dijkstra observed that "*the GOTO statement as it stands is just too primitive; it is too much an invitation to make a mess of one's program [...] the GOTO statement should be abolished from all higher-level programming languages*" [37]. In the computer programming community, Dijksta's observation started a controvert debate against the expressivity power of GOTO statements [38]–[40]. Finally, the community followed Dijksta's advice, and modern software no longer contains GOTO statements. However, we still can find GOTO statements everywhere in the form of *transitions* in FSMs, and therefore, CHSs.

The robotics community identified similarities in the desired behaviors for NPCs and robots. In particular, both NPCs and robots act in an unpredictable and dynamic environment; both need to encode different behaviors in a hierarchy, both need

Manuscript received 26 April 2021; revised 15 September 2021; accepted 21 October 2021. Date of publication 16 December 2021; date of current version 8 August 2022. This work was supported by the European Union's Horizon 2020 research and innovation programme under Grant Agreement 732410 of the SCOPE project, in the form of financial support to third parties of the RobMoSys project. This paper was recommended for publication by Associate Editor F. Amigoni and Editor W. Burgard upon evaluation of the reviewers' comments. *(Corresponding author: Michele Colledanchise.)*

The authors are with the Humanoids Sensing and Perception Laboratory, Center for Robotics and Intelligent Systems, Istituto Italiano di Tecnologia, 30 16163 Genoa, Italy (e-mail: michele.colledanchise@iit.it; lorenzo.natale@iit.it).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TRO.2021.3125863.

Digital Object Identifier 10.1109/TRO.2021.3125863

a compact representation for their behaviors. Quickly, BTs became a modular, flexible, and reusable alternative over FSM to model robot behaviors [41]. Moreover, the robotic community showed how BTs generalize successful robot control architectures such as the subsumption architecture and the teleo-reactive paradigm [1]. Using BTs, the designer can compose simple robot behaviors using the so-called *control flow nodes:* Sequence, Fallback, Decorator, and Parallel. As mentioned, the Parallel composition of independent behaviors can arise several concurrency problems in any modeling language, and BTs are no exception. However, the parallel composition of BTs remains less sensitive to dimensionality problems than classical FSMs [1].

Another similarity between computer programming and robot behavior design lies in the concurrent execution of multiple tasks. A computer programmer adopts synchronization techniques to achieve *execution synchronization*, where a process has to wait until another process provides the necessary data, or *data synchronization*, where multiple processes have to use or access a critical resource and a correct synchronization strategy ensures that only one process at a time can access them [42]. The solutions adopted in concurrent programming made a tremendous impact on software development. Another desired quality of a process, particularly in real-time systems, is the *predictability*, that is, the ability to ensure the execution of a process regardless of outside factors that will jeopardize it. In other words, the application will behave as intended in terms of functionality, performance, and response time.

Nowadays, robot software follows a modular approach, in which computation and control use concurrent execution of interconnected modules. This philosophy is promoted by middlewares like ROS and has become a defacto standard in robotics. The presence of concurrent behaviors requires facing the same issues affecting concurrent programming, which deals with the execution of several concurrent processes that need to be synchronized to achieve a task or simply to avoid being in conflict with one another. In this context, proper synchronization and resource management become beneficial to achieve faster and reproducible behaviors, especially at the developing stage, where actions may run at a different speed in the real world and in a simulation environment. Increasing predictability reduces the difference between simulated and real-world robot executions and increases the likelihood of task completion within a given time constraint. We believe that proper parallel task executions will bring benefits in terms of efficiency and multitasking to BTs in a similar way as in computer programming.

Nontechnical specifications may also require concurrent or predictable behaviors. For example, the human–robot interaction (HRI) community stressed the importance of synchronized robots' behaviors in several contexts. The literature shows evidence of more "believable" robots behaviors when they exhibit contingent movements [43] (e.g., synchronized gaze and arm movement), coordinated robots and human movements [44] (e.g., a rehabilitation robot moves at the patient's speed), and coordinated gestures and dialogues [45] (e.g., the robot's gesture synchronized with dialogues).

In this article, we extend our previous works [46], [47], we define concurrent BTs (CBTs), where nodes expose information regarding progress and resource used, we also define and implement two new control flow nodes that allow progress and data synchronization techniques and show how to improve behavior predictability. In addition, we introduce measures to assess execution performance and show how design choices affect them. To clarify what we mean by these concepts, we consider the following task: *A robot has to follow a person*. The robot's behavior can be encoded as the concurrent execution of two subbehaviors: *navigation* and *gazing*. However, the navigation behavior is such that, whenever the robot gets lost, it moves the head, looking for visual landmarks to localize itself. Fig. 1 encodes the BT of this behavior. At this stage, the semantic of BT is not required to understand the problem. Note how, whenever the robot gets lost, two actions require the use of the head: the actions *Look for Landmarks* and *Look for Person*. To avoid conflicts, we have to modify to be as in Fig. 1. However, such BT goes against the separation of roles as the BT designer needs to know beforehand the actions' resources. In this article, we propose two control flow nodes that allow the synchronization of such BT in a less invasive fashion, as in Fig. 1. The concurrent execution of legacy BTs represents another example of such a synchronization mechanism. Clearly, the design of a single action that performs both tasks represents a "better" synchronized solution. However, creating the single action for composite behaviors jeopardizes the advantages of BTs in terms of modular and reusable behavior.

To summarize, in this article, we provide an extension of our previous work [46] and [47], where the new results are the following:

1) We moved the synchronization logic from the parallel node to the decorator nodes.
2) We provide reproducible experimental validation on simulated data.
3) We provide experimental validation on three real robots.
4) We compared our approach with two alternative task synchronization techniques.
5) We provide the code to extend an existing software library of BTs, and its related GUI, to encode the proposed synchronization nodes.
6) We provide a theoretical discussion of our approach and identify the assumptions under which the property on BTs are not jeopardized by the synchronization.

The outline of this article is as follows. We present the related work and compare it with our approach in Section II. We overview the background in BTs in Section III. We present the first contribution of this article on BT synchronization in Section IV. Then we present the second contribution on performance measures in Section V. In Section VI, we provide experimental validation with numerical experiments to gather statistically significant data and compare our approach with existing ones. We made these experiments reproducible. We also validated our approach on real robots in three different setups to show the applicability of the approach to real problems. We describe the software library we developed, and we refer to the online repository in Section VII. We study the new control nodes from a theoretical standpoint and study how design choices affect the performances in Section VIII. Section IX concludes this article.

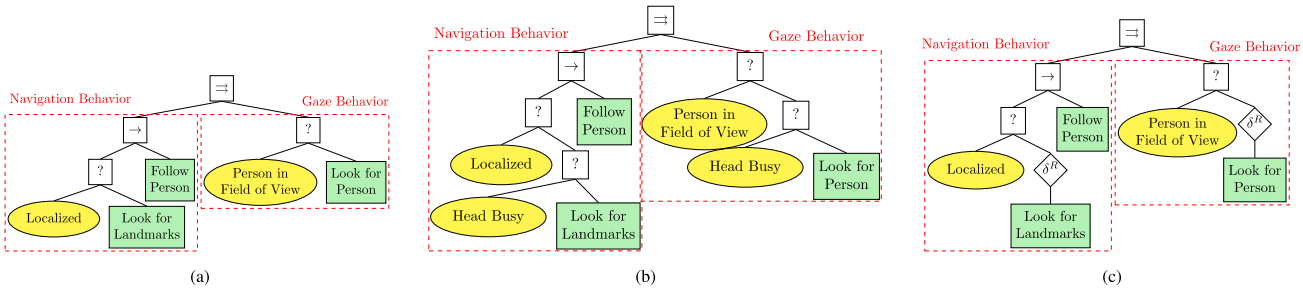

Fig. 1. Example of flawed and correct concurrent BT execution. The gaze and navigation behaviors are executed in parallel. (a) Flawed BT execution. The conflicting actions Look for Person and Look for Landmarks may be executed concurrently. (b) Correct BT execution using the classical BT nodes. (c) Correct BT execution using the proposed BT nodes.

II. Related Work

This section shows how BT designers in the community exploit the parallel composition, and we highlight the differences with the proposed approach. We do not compare our approach with generic scheduling algorithms [48], as our interest lies in the concurrent behaviors encoded as BTs.

The parallel composition has found relatively little use, compared to the other compositions, due to the intrinsic concurrency issues similar to the ones of computer programming, such as race conditions and deadlocks. Current applications that make use of the parallel composition work under the assumption that sub-BTs lie on orthogonal state spaces [2], [49] or that sub-BTs executed in parallel have a predefined priority assigned [50] where, in conflicting situations, the sub-BTs with the lower priority stops. Other applications impose a mutual exclusion of actions in sub-BTs whenever they have potential conflicts (e.g., sending commands to the same actuator) [1] or they assume that sub-BTs that are executed in parallel are not in conflict by design.

The parallel composition found large use in the BT-based task planner *A Behavior Language* (ABL) [51] and in its further developments. ABL was originally designed for the game *Façade*, and it has received attention for its ability to handle planning and acting at different deliberation layers, in particular, in real-time strategy games [50]. ABL executes sub-BTs in parallel and resolves conflicts between multiple concurrent actions by defining a fixed priority order. This solution threatens the reusability and modularity of BTs and introduces a latent hierarchy in the BT.

The parallel composition found use also in multirobot applications, both with centralized [52] and distributed fashions [53], [54], resulting in improved fault tolerance and other performances. The parallel node involves multiple robots, each assigned to a specific task using a task-assignment algorithm. A task-assignment algorithm ensures the absence of conflicts.

None of the existing work in the BT literature adequately addressed the synchronization issues that arise when using a parallel BT node. They assume or impose strict constraints on the actions executed and often introduce undesired latent hierarchies difficult to debug.

A recent work [55] proposed BTs for executing actions in parallel, even when they lie on the same state space (e.g., they use the same robot arm). The authors implement the coordination mechanism with a BT that activates and deactivates motion primitives based on their preconditions. Such a framework avoids that more actions access a critical resource concurrently. In our work, we are interested in synchronizing the progress of actions that a BT can execute concurrently.

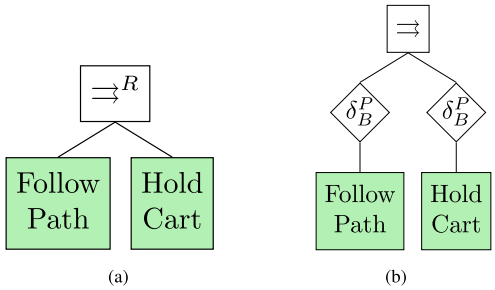

Fig. 2. BT synchronization using the previous [46] (left) and the proposed formulation (right). (a) BTs synchronization using the previous formulation. (b) BTs synchronization using the proposed formulation.

We address the issues above by defining BT nodes that expose information regarding progress and resource uses. We define an absolute and relative synchronized parallel BT node execution and a resource handling mechanism. We provide a set of statistically meaningful experiments and real-robot executions. We also provide an extension to the software library to obtain such synchronizations and real-robot examples. This makes our article fundamentally different than the ones presented above and the BT literature.

In our previous work [46], [47], we extend the semantic of the parallel node to allow synchronization. Fig. 2(a) shows an example of a synchronized BT using our such approach. In this article, we go beyond our previous work by moving the synchronization logic inside a decorator node, as Fig. 2(b). That allows the synchronization to deeper branches of the BT, as in Fig. 3, and multiple cross synchronization. In Section VI, we will also show a synchronization experiment possible only with the new semantic.

III. Background

This section briefly presents the classical and the state-space formulation of BTs. A detailed description of BTs is available in the literature [1].

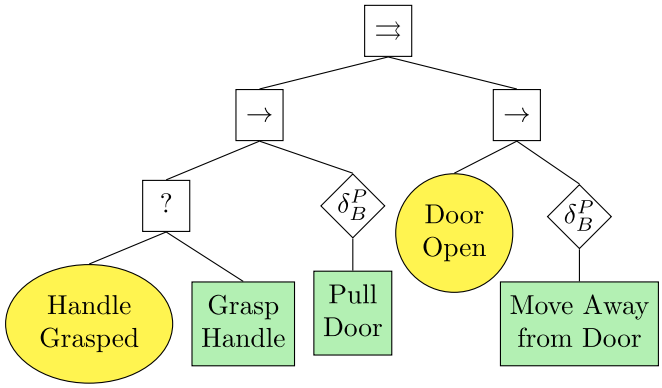

Fig. 3. More complex version of the BT for Example 1 allowed by the new formulation only.

Algorithm 1: Pseudocode of a Sequence Operator With N Children.

```
1 Function Tick()
2     for i ← 1 to N do
3         childStatus ← child(i).Tick()
4         if childStatus = Running then
5             return Running
6         else if childStatus = Failure then
7             return Failure
8     return Success
```

A. Classical Formulation of Behavior Trees

A BT is a graphical modeling language that represents actions orchestration. It is a directed rooted tree where the internal nodes represent behavior compositions and leaf nodes represent actuation or sensing operations. We follow the canonical nomenclature for root, parent, and child nodes.

The children of a BT node are placed below it, as in Fig. 3, and they are executed in the order from left to right. The execution of a BT begins from the root node. It sends *ticks*, which are activation signals, with a given frequency to its child. A node in the tree is executed if and only if it receives ticks. When the node no longer receives ticks, its execution stops. The child returns to the parent a status, which can be either *Success*, *Running*, or *Failure* according to the node's logic. Below we present the most common BT nodes and their logic.

In the classical representation, there are four operator nodes (Fallback, Sequence, Parallel, and Decorator) and two execution nodes (Action and Condition). There exist additional operators, but we will not use them in this article.

Sequence: When a sequence node receives ticks, it routes them to its children in order from the left. It returns Failure or Running whenever a child returns Failure or Running, respectively. It returns Success whenever all the children return Success. When child i returns Running or Failure, the Sequence node does not send ticks to the next child (if any) but keeps ticking all the children up to child i. The Sequence node is graphically represented by a square with the label "→", as in Fig. 3, and its pseudocode is described in Algorithm 1.

Algorithm 2: Pseudocode of a Fallback Operator With N Children.

```
1 Function Tick()
2     for i ← 1 to N do
3         childStatus ← child(i).Tick()
4         if childStatus = Running then
5             return Running
6         else if childStatus = Success then
7             return Success
8     return Failure
```

Algorithm 3: Pseudocode of a Parallel Operator With N Children.

```
1 Function Tick()
2     forall i ← 1 to N do
3         childStatus[i] ← child(i).Tick()
4     if Σ_{i:childStatus[i]=Success} = M then
5         return Success
6     else if Σ_{i:childStatus[i]=Failure} > N − M then
7         return Failure
8     else
9         return Running
```

Fallback: When a Fallback node receives ticks, it routes them to its children in order from the left. It returns a status of Success or Running whenever a child returns Success or Running respectively. It returns Failure whenever all the children return Failure. When child i returns Running or Success, the Fallback node does not send ticks to the next child (if any) but keeps ticking all the children up to the child i. The Fallback node is represented by a square with the label "?," as in Fig. 3, and its pseudocode is described in Algorithm 2.

Parallel: When the Parallel node receives ticks, it routes them to all its children. It returns Success if $M \geq N$ children return Success, it returns Failure if more than $N - M$ children return Failure, and it returns Running otherwise. The Parallel node is graphically represented by a square with the label "⇉", as in Fig. 3, and its pseudocode is described in Algorithm 3.

Decorator: A Decorator node represents a particular control flow node with only one child. When a Decorator node receives ticks, it routes them to its child according to custom-made policy. It returns to its parent a return status according to a custom-made policy. The Decorator node is graphically represented as a rhombus, as in Fig. 3. BT designers use decorator nodes to introduce additional semantic of the child node's execution or to change the return status sent to the parent node.

Action: An action performs some operations as long as it receives ticks. It returns Success whenever the operations are completed and Failure if the operations cannot be completed. It returns Running otherwise. When a running Action no longer receives ticks, its execution stops. An Action node is graphically represented by a rectangle, as in Fig. 3, and its pseudocode is described in Algorithm 4.

Algorithm 4: Pseudocode of a BT Action.

```
1 Function Tick()
2     DoAPieceOfComputation()
3     if action-succeeded then
4         return Success
5     else if action-failed then
6         return Failure
7     else
8         return Running
```

Algorithm 5: Pseudocode of a BT Condition.

```
1 Function Tick()
2     if condition-true then
3         return Success
4     else
5         return Failure
```

Condition: Whenever a Condition node receives ticks, it checks if a proposition is satisfied or not. It returns Success or Failure accordingly. A Condition is graphically represented by an ellipse, as in Fig. 3, and its pseudocode is described in Algorithm 5.

B. Control Flow Nodes With Memory

To avoid the unwanted re-execution of some nodes, and save computational resources, the BT community developed control flow nodes with memory [33]. Control flow nodes with memory keep stored which child has returned Success or Failure, avoiding their re-execution. Nodes with memory are graphically represented with the addition of the symbol "$*$" as superscript (e.g., a Sequence node with memory is graphically represented by a box with the label "$\rightarrow^*$"). The memory is cleared when the parent node returns either Success or Failure so that, at the next activation, all children are reconsidered. Every execution of a control flow node with memory can be obtained employing the related nonmemory control flow node using auxiliary conditions and shared memory [1]. Provided a shared memory, these nodes become syntactic sugar.

C. Asynchronous Action Execution

Algorithm 4 performs a step of computation at each tick. It implements an action execution that is synchronous to the ticks' traversal. However, in a typical robotics system, action nodes control the robot by sending commands to a robot's interface to execute a particular skill, such as an arm movement or a navigation skill; these skills are often executed by independent components running on a distributed system. Therefore, the action execution gets delegated to different executables that communicate via a middleware.

As discussed in the literature [1], [56], the designer needs to ensure that the skills running in the robot independently from the BT get properly interrupted when the corresponding action no longer receives ticks.

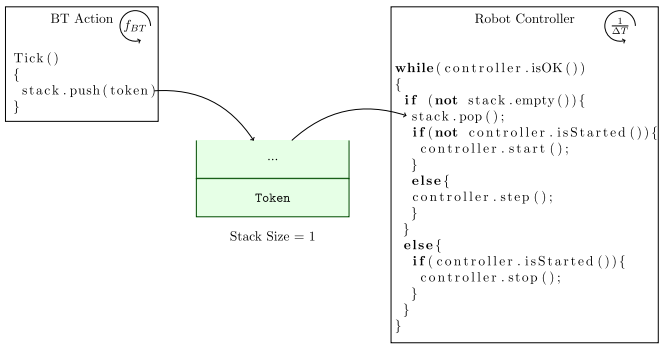

Fig. 4. Example of asynchronous external action execution. f_{BT} is the tick frequency and ΔT is the quantum period.

To support the preemption and synchronization, BT designers split the actions execution in smaller steps, each executed within a *quantum*, that is, a time window during which the action gets executed by the robot asynchronously with respect to the BT. During this time, the action cannot be interrupted. The action starts when the first tick is received, and it proceeds for another quantum only when the next tick is received. At the end of each quantum, a running action yields control back to the BT. This logic resembles process scheduling, where a scheduler provides computing time, to a process, and then it takes back control to choose the next process to run. Fig. 4 shows an example of how a BT action interacts with an external executable that controls the robot. The figure depicts two threads, one that ticks the action node (within the BT executable) and one that controls the robot (within an external executable). When the action node receives a tick from its parent, it pushes a token onto a stack, shared with the external executable that controls the robot. Such executable controls the robot if and only if there is a token in the stack. This behavior also is outlined in the algorithm in Fig. 4. The executable checks the stack periodically, if there is a token in the stack, it consumes it, and it executes a control step. If the stack is empty, the executable halts the controller execution. If the BT tick frequency is faster than the controller quantum (e.g., twice the thread's frequency), this mechanism ensures that the controller operates without interruptions, but it also guarantees that the controller is halted when ticks are no longer dispatched to the action node without delay (this is achieved using the size of the stack equal to one).

It is clear that, in BTs, the tick frequency plays an important role in action preemption. To allow "fast" preemption, the quantum of actions should be short and the tick frequency should be high. Intuitively, a blocking action, which does not allow to be interrupted throughout its entire execution, will continue to take control of the robot also if it no longer receives ticks. BTs orchestrate behaviors at a relatively high level of abstraction. In general, to avoid preemption delay, the time spanned between a tick and the next one must be shorter than the smallest action quantum in the BT. In our experience, a quantum of $\Delta T \approx 100ms$ (i.e., an update frequency of $10\,Hz$) and a tick frequency of $20\,Hz$

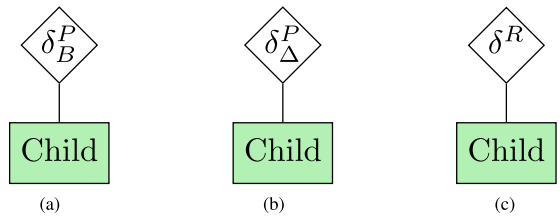

Fig. 5. Graphical representation of a synchronization decorator nodes. (a) Absolute synchronization. B indicates the set of barriers. (b) Relative synchronization. Δ indicates the threshold value. (c) Resource synchronization decorator node.

represents a good trade-off between action responsiveness and required ticks traversal frequency.

The interaction between the BT and the executable is designed to ensure that the robot controller continues to operate if the BT ticks the action node without interruptions. Otherwise, i.e., if no ticks are received within the assigned quantum, the controller is halted.

IV. Concurrent BTs

This section introduces the first contribution of the article. We present the concurrent BTs (CBTs), an extension to classical BTs with the addition of the execution progress and the resource allocated in the formulation. We extend our previous work on parallel synchronization of BTs [46], [47] employing decorator nodes that allow progress and resource synchronization. Here, we define these nodes by their pseudocode. We provide the source code for some of the examples provided.[1]

In Section VIII, we provide the formal definition and the state-space formulation of the nodes. We also prove that, under some assumptions, the proposed nodes do not jeopardize the BT properties.

Concurrent BTs: A CBT is a BT whose nodes expose information about the execution progress $p(x_k)$ and the resource required $Q(x_k)$ and priority $\rho(x_k)$ at system's state $x_k \in \mathbb{R}^n$. In addition, the nodes contain the user-defined function $g(x_k)$ that represents the priority increase whenever the execution of a node gets denied by a resource not available; we will present the details in this section. In Section VIII, we provide the formal definition of CBTs and the formulation of the Sequence and Fallback composition.

ProgressSynchronization Decorator: When a ProgressSynchronization Decorator receives a tick, it ticks its child if the child's progress at the current state $p(x_k)$ is lower than the current barrier $b(x_k)$. The decorator returns to the parent Success if the child returns Success, it returns to the parent Failure if the child returns Failure. It returns Running otherwise. The ProgressSynchronization Decorator node is graphically represented by a rhombus with the label "δ^P_b", as in Fig. 5(a), and its pseudocode is described in Algorithm 6.

We will calculate the barrier $b(x_k)$ either in an absolute or relative fashion, as we will show in Sections IV-A and IV-B.

[1][Online]. Available: https://github.com/miccol/tro2021-code

Algorithm 6: Pseudocode of a ProgressSynchronization Decorator.

```
1 Function Tick()
2     if child.p(x_k) ≤ b(x_k) then
3         childStatus ← child.Tick()
4         return childStatus
5     return Running
```

Resource Synchronization Decorator: When a ResourceSynchronization Decorator receives a tick, it ticks its child if the resources required by the child i, $Q_i(x_k)$, are either available or assigned to that child already. When the decorator ticks a child, it also assigns all the resources in $Q_i(x_k)$ to that child. Whenever the child no longer requires a resource in $Q_i(x_k)$, such resource gets released. The decorator returns to the parent Success if the child returns Success, It returns to the parent Failure if the child returns Failure. It returns Running if either the child return running or if the child is waiting for a resource. R is the set of all the resources of the system. The decorator keeps also a priority value for the subtree accessing a resource, to avoid starvation, as we prove it in Section VIII. Whenever the decorator receives a tick and does not send it to the child (as the resources are not available), the priority value evolves according to the $g(x_k)$. In Section VI, we will show two examples that highlight how the choice of the function g avoids starvation. The BT keeps track of the node currently using a resource q, via the function $\alpha(q)$. All the resource decorator nodes share the value of such function.

The ResourceSynchronization Decorator node is graphically represented by a rhombus with the label "δ^R", as in Fig. 5(c). Algorithm 7 describes its pseudocode, in particular, for each resource q required by the decorator's child (Line 2), if the resource results assigned to another child (Line 3), then the priority of the child to get the resource q increases according to the function g. The algorithm then assigns the resources to the child with the highest priority (Lines 7–9) and releases the child's resources if either it no longer requires it (Lines 10–11).

Remark 1: We are not interested in a scheduler that fairly assigns the resources as it is done, for example, in the Operating Systems. The designer may implement a fair scheduling policy and encode it in the function g. However, if an action has always higher priority than another one to get a resource, this should be modeled via a Sequence or Fallback composition.

A. Absolute Progress Synchronization

A BT achieves an absolute progress synchronization by setting, *a priori*, a finite ordered set $\mathcal{B}$ of values for the progress. These values are used as *barriers* at the task level [42]. Whenever a child of an AbsoluteProgressSync Decorator has the progress equal to or greater than a progress barrier in $\mathcal{B}$ it no longer receives ticks until all the other nodes whose parent is an instance of such decorator have the progress equal to or greater than the barrier's value. We now present a use case example for the absolute progress synchronization, taking inspiration from the literature [57], [58].

Algorithm 7: Pseudocode of a ResourceSynchronization Decorator.

```
1  Function Tick()
2      for q in child.Q(x_k) do
3          if (α(q) not = child) and (α(q) not ∅) then
4              child.ρ(x_k) ← child.ρ(x_{k−1}) + g(x_k)
5              return Running
6      for q in R do
7          if q in child.Q(x_k) then
8              if α(q) not = child
                   child.ρ(x_k) > α(q).ρ(x_k) then
9                  α(q) ← child
10         else if α(q) = child then
11             α(q) ← ∅
12     childStatus ← child.Tick()
13     return childStatus
```

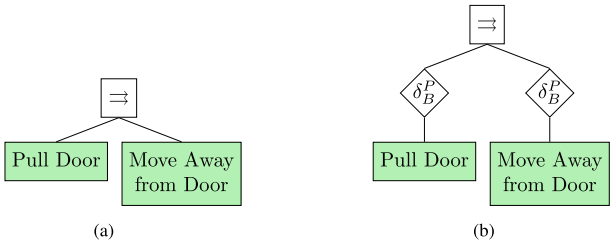

Fig. 6. BT encoding the desired behavior of Example 1. (a) Without synchronization. (b) With synchronization.

Example 1 (Absolute): A robot has to pull a door open. To accomplish this task, the robot must execute two behaviors concurrently: an arm movement behavior to pull the door open and a base movement behavior to make the robot move away from the door while this opens, as the BT in Fig. 6(a).

The progress profile of the two sub-BTs, "Pull Door" $\mathcal{T}_1$ and "Move Away from Door" $\mathcal{T}_2$, holds the equations as follows:

$$p_i(x_k) = \begin{cases} 0 & \text{if } k = 0 \\ p_i(x_{k-1}) + a_i, & \text{otherwise} \end{cases} \tag{1}$$

with $a_1 = 0.015$ (Pull Door), $a_2 = 0.01$ (Move Away from Door). The equations describe a linear progress profile for both behaviors, with the action "Pull Door" faster than the action "Move Away from Door." However, to ensure that the task is correctly executed, the robot must execute the two behaviors above in a synchronized fashion. The BT in Fig. 6(b) encode such synchronized behavior, with the following barriers:

$$B = \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}. \tag{2}$$

Fig. 7 shows the progress profiles of the actions with and without synchronization. We see how, in the synchronized case, the arm movement keeps stopping to wait for the base movement at the points of executions defined in the barrier.

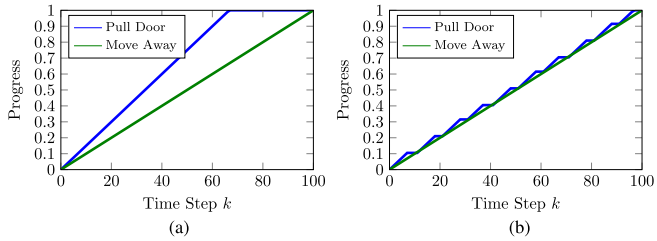

Fig. 7. Progress profiles of the actions of Example 1. (a) Without synchronization. (b) With synchronization.

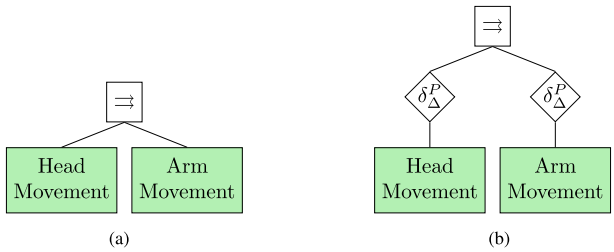

Fig. 8. BT encoding the desired behavior of Example 2. (a) Without synchronization. (b) With synchronization.

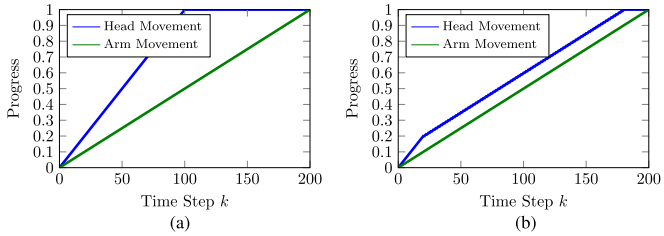

Fig. 9. Progress profiles of the actions of Example 2. (a) Without synchronization. (b) With synchronization.

B. Relative Progress Synchronization

In this case, synchronization does not follow a common progress indicator, but it is relative to another node's execution. A BT achieves a relative progress synchronization by setting *a priori* a threshold value $\Delta \in [0, 1]$. Whenever a child of a RelativeProgressSync Decorator exceeds the minimum progress, among all the other nodes whose parent is an instance such decorator, by Δ, it no longer receives ticks.

We now present a use case example for the relative progress synchronization, taking inspiration from the literature [43]. We provide an implementation this example in Section VI.

Example 2 (Relative): A service robot has to give directions to visitors to a museum. To make the robot's motions look natural, whenever the robot gives a direction, it points with its arm and the head to that direction as in the BT in Fig. 8(a).

The progress profile of the two sub-BTs, "Head Movement" $\mathcal{T}_1$ and "Arm Movement" $\mathcal{T}_2$, holds the equations as follows:

$$p_i(x_k) = \begin{cases} 0 & \text{if } k = 0 \\ p_i(x_{k-1}) + a_i, & \text{otherwise} \end{cases} \tag{3}$$

with $a_1 = 0.01$ (Arm), $a_2 = 0.05$ (Head). Fig. 9(b) shows the progress profile of the sub-BTs.

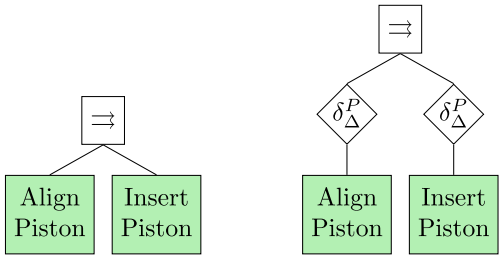

Fig. 10. BT encoding the desired behavior of Example 3. (a) Without synchronization. (b) With synchronization.

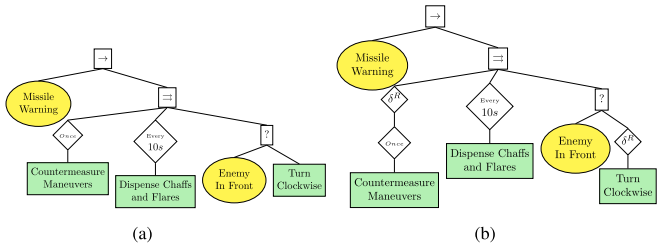

Fig. 11. BT encoding the desired behavior of Example 4, adapted from [59]. (a) Without synchronization. (b) With synchronization.

The equations describe a linear progress profile for both behaviors, with the action "Move Head" faster than the action "Move Arm." However, the arm and head may require different times to perform the motion, according to the direction to point at. Hence, to look natural, the head movement must follow the arm movement to avoid the unnatural behavior where the robot looks first to a direction and then points at it, or the other way round. The BT in Fig. 8(b) encode such synchronized behavior, with $\Delta = 0.1$

Fig. 9 shows the progress profiles of the actions. We see how the head movement stops when its progress surpasses the arm movement's progress by 0.1, around time step $k = 10$.

Perpetual Actions: We can use the relative synchronized parallel node also to impose coordination between *perpetual* actions, (i.e., an action that, even in the ideal case, does not have a fixed duration, hence a progress profile), as in the following example taken from the literature [55]. We will present an implementation of the example above in Section VI.

Example 3 (Perpetual actions): An industrial manipulator has to insert a piston into a cylinder of a motor block. It is an instance of a typical peg-in-the-hole problem with the additional challenge of the freely swinging piston rod. To correctly insert the piston, the latter must be kept aligned during the insertion into the cylinder.

We can describe this behavior as a parallel BT composition of two sub-BTs: one for inserting the piston and one for keeping the piston aligned with the cylinder as in Fig. 10. The Insert Piston action stops when the end-effector senses that the piston hits the cylinder's base, hence its progress cannot be computed beforehand. Since the inserting behavior and the alignment behavior are executed concurrently, the robot may insert the piston too fast, resulting in a collision between the piston and the cylinder's edge.

Fig. 10(b) shows a BT of a synchronized execution, where the progress of the piston insertion (Insert Piston action) has only two values—0 and 1. It equals 1 whenever the piston is being inserted and 0 otherwise. Similarly, for the alignment behavior (Align Piston action). The insertion behavior stops while the robot is aligning the piston.

Remark 2: In real-world scenarios, we can compute the progress either in an open-loop (i.e., at each tick received it increments the progress) or in a closed-loop (i.e., using the sensors to compute the actual progress) fashion.

C. Resource Synchronization

This section shows how CBTs can execute multiple actions in parallel without resource conflicts. This synchronization becomes useful when executing in parallel BTs that have some actions in common, as shown in Example 4, adapted from the BT literature. This often happens whenever we want to execute concurrently existing BTs.

Example 4: The BT in Fig. 11(a) shows a BT for a missile evasion tactic, taken from the literature [59]. The BT has three sub-BTs that run in parallel: "Turn on Countermeasure Maneuvers," "Countermeasure Maneuvers once," "Dispense Chaff and Flares Every 10 Seconds," "Turn Clockwise if an Enemy on a Range."

The actions "Countermeasure Maneuver" and "Turn Clockwise" run in parallel and both use the aircraft's actuation. There are cases in which both actions receive ticks, resulting in possible conflicts. We can use the resource decorator node to avoid such conflicts, as in the BT in Fig. 11(b).

The authors of [59] did not address the concurrency issue above. However, taking advantage of the composability of BTs, we did the modification easily.

D. Improving Predictability

We can use progress synchronization to impose a given progress profile constraint. The idea is to define an artificial action with the desired progress profile (over time) defined *a priori* and putting it as a child of an absolute synchronized parallel node with the actions whose progress is to be constrained. However, since we can only stop actions (i.e., BTs have no means to speed up actions), we can only define such artificial action as progress upper bound. This type of progress profile creation may become very useful at the developing stage since the actions may run at a different speed in the real world and in a simulation environment. Improving predictability reduces the difference between simulated and real-world robot execution.

Example 5: A robot has to move its arm following a sigmoid profile (i.e., the movements are first slow, then fast, then slow again). However, the manipulation action is designed to follow a linear profile (i.e., same movement's speed throughout the execution). To impose the desired profile we create the action "Profile" that models the sigmoid and we impose a progress synchronization with the manipulation action. The BT in Fig. 12 shows the BT to encode this task.

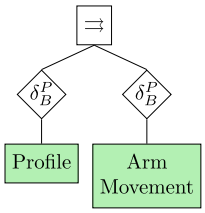

Fig. 12. BT encoding the desired behavior of Example 5.

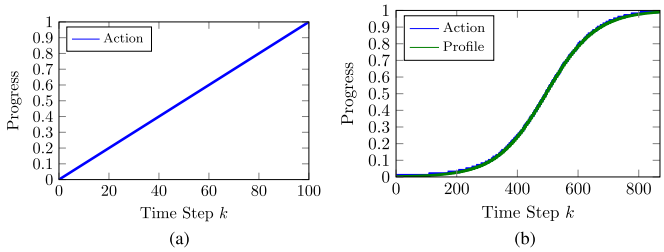

Fig. 13. Progress profiles of Example 5 with and without synchronization. (a) Without synchronization. (b) With synchronization.

Fig. 13 shows the progress profiles of the action with and without the progress profile imposition. Note how the action's progress profile changed without editing the action. However, this was possible as the action, originally, has a faster progress profile than the desired one, as we have no nonintrusive means to speed up actions.

V. Synchronization Measures

This section presents the second contribution of the article. We define measures for the concurrent execution of BTs used to establish execution performance. We show measures for both progress synchronization and predictability. In Section VI, we show how the design choices for relative and absolute parallel nodes affect the performance.

A. Progress Synchronization Distance

Definition 1: Let N be the number of nodes that have as a parent the same instance of a progress decorator node, the progress distance at state x_k is defined as

$$\pi(x_k) \triangleq \sum_{i=1}^{N}\sum_{j=1}^{N} \frac{|p_i(x_k) - p_j(x_k)|}{2} \tag{4}$$

where $p_i \in [0,1]$ is the progress of the ith child, as in Section IV above.

We sum the progress difference for each pair of nodes that have as parent the same instance of a progress decorator node. We divide by 2 to avoid double count the differences. We use the absolute difference instead of a, e.g., squared difference to assign equal weight to the spread of the progresses.

Intuitively, a small progress distance results in high performance for both relative and absolute progress synchronization.

B. Predictability Distance

A useful method to measure predictability is to set the desired progress value and compute the average variation from the expected and the true time instant in which the action has a progress that is closest to the desired one. We can use this measure to assess the deviation from the ideal execution.

Definition 2: Given a progress value $\bar{p} \in [0,1]$, a time step $\tilde{t}_k \triangleq \operatorname{argmin}_{t_k}(p(x(t_k)) - \bar{p})$, and $\hat{t}_k$ be the time instance when $p(x(t_k))$ is expected to be equal to $\bar{p}$. The time predictability distance relative to progress $\bar{p}$ is defined as

$$P(\bar{p}) \triangleq |\tilde{t}_k - \hat{t}_k|. \tag{5}$$

Remark 3: A node may not obtain the exact desired progress value as the progress may be defined at discrete points of execution. Hence, in Definition 2 we compute the difference between the desired progress value ad the closest one obtained.

VI. Experimental Validation

We conducted numerical experiments that allow us to collect statistically significant data to study how the design choices affect the performance measures defined in Section V and to compare our approach against other solutions. We made the source code available online for reproducibility.[2] We also conducted experiments on real robots to show the applicability of our approach in the real world. We made available online a video of these experiments.[3]

A. Numerical Experiments

We are ready to show how the number of barriers in $\mathcal{B}$ (for absolute synchronization) and the threshold value Δ (for relative synchronization) affect the performance, computed using the measures defined in Section V. For illustrative purposes, we define custom-made actions with different progress profiles. To collect statistically significant data, we ran the BT of each experiment 10,000 times; we use boxplots to compactly show the minimum, the maximum, the median, and the interquartile range of the measures proposed. Each experiment starts with the progress of actions equal to 0 and ends when all the actions progress reach 1.

How the number of progress barriers affects the performance of absolute synchronization. We now present an experiment that highlights how the number of progress barriers in $\mathcal{B}$ affects the performance of absolute synchronization.

Experiment 1: Consider the BT in Fig. 14(a) where the progress decorator implements an absolute synchronization with equidistant barriers (i.e., a barrier at each $\frac{1}{|\mathcal{B}|}$ progress) and the sub-BTs are such that the progress profile of each $\mathcal{T}_i$ holds (6) as follows:

$$p_i(x_k) = \begin{cases} 0 & \text{if } k = 0 \\ p_i(x_{k-1}) + a_i + \omega_i(x_k), & \text{otherwise} \end{cases} \tag{6}$$

[2][Online]. Available: https://github.com/miccol/tro2021-code
[3][Online]. Available: https://youtu.be/zCBuTYogb_U

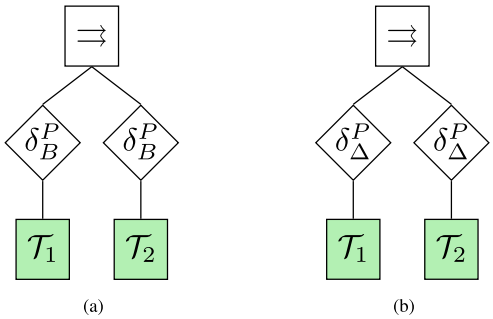

Fig. 14. BT used for Experiments 1 and 2. (a) Experiments 1. (b) Experiments 2.

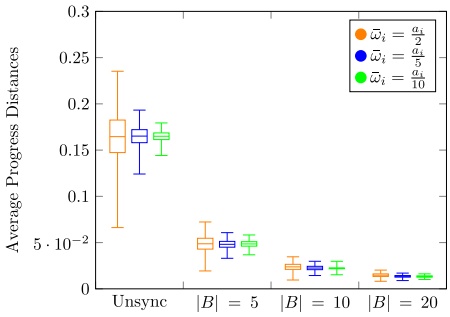

Fig. 15. Boxplot of the progress distances of Experiment 1 with different numbers of barriers $|\mathcal{B}|$ and different values of $\bar{\omega}$. $|\mathcal{B}| = 0$ corresponds to the unsynchronized execution.

with $a_1 = 0.03$, $a_2 = 0.02$, and $\omega_i(x_k) \in [-\bar{\omega}, \bar{\omega}]$ a random number, sampled from an uniform distribution, in the interval $[-\bar{\omega}, \bar{\omega}]$.

The model above describes an action whose progress evolves linearly with a fixed value (a_i) and with some disturbance ($\omega_i(x_k)$), modeling possible uncertainties in the execution that affect the progress.

Fig. 15 shows the results of running 10,000 times the BT in Experiment 1 in different settings and computing the average progress distance throughout the execution. We observe better performance with a large number of barriers and smaller $\bar{\omega}$. This shows that a higher number of barriers prevents the progress of the actions to differ from each other (see Algorithm 6). Note also that the synchronization yields a reduced variance even with large values of ω.

Remark 4: In Experiment 1, we consider equidistant progress barriers. We expect similar results with nonequidistant barriers, except for the corner case in which all the barriers are agglomerated in a specific progress value.

How the threshold value affects the performance of relative synchronization. We now present an experiment that highlights how the value of Δ affects the performance of relative synchronization.

Experiment 2: Consider the BT in Fig. 14(b) where the progress decorator implements a relative synchronization and the sub-BTs are such that the progress profile of each $\mathcal{T}_i$ holds

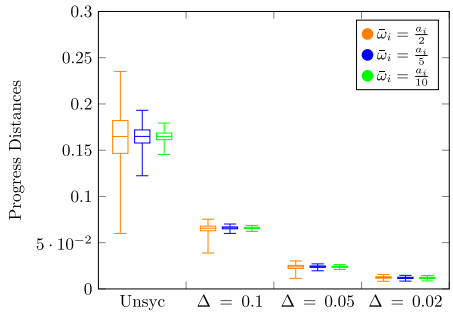

Fig. 16. Boxplot of the progress distances of Experiment 2 with different values for Δ and $\bar{\omega}$. $\Delta = 1$ corresponds to the unsynchronized execution.

(7) as follows:

$$p_i(x_k) = \begin{cases} 0 & \text{if } k = 0 \\ p_i(x_{k-1}) + a_i + \omega_i(x_k), & \text{otherwise} \end{cases} \quad (7)$$

with $a_1 = 0.03$, $a_2 = 0.02$, and $\omega_i(x_k) \in [-\bar{\omega}, \bar{\omega}]$ a random number, sampled from an uniform distribution, in the interval $[-\bar{\omega}, \bar{\omega}]$.

The model above describes an action whose progress evolves linearly with a fixed value (a_i) and with some disturbance ($\omega_i(x_k)$), modeling possible uncertainties in the execution that affect the progress.

Fig. 16 shows the results of running 10,000 times the BT in Experiment 2 in different settings and computing the average progress distance throughout the execution. We observe that the performance increases with a smaller Δ and decreases with a larger $\bar{\omega}$. This shows that a smaller Δ prevents the progress of the actions to differ from each other (see Algorithm 6). Note also that the synchronization yields a reduced variance even with large values of ω.

Remark 5: The synchronization may deteriorate other desired qualities. For example, since actions are waiting for one another, the overall execution may be slower than the slowest action. Moreover, a small value for Δ or a larger number of barriers can result in highly intermittent behaviors.

Remark 6: The decorators can be placed in different parts of the BT and not as direct children of a parallel node, as shown in Section II.

Remark 7: A single action that performs both tasks represents a better synchronized solution. However, for reusability purposes or for the separation of concerns, the designer may want to implement the behavior using two separated actions.

Progress synchronization comparison: We now present an experiment that compares the synchronization performance. We compare our approach with three different alternative ones: One using elements from the C++11's standard library,[4] as it is the programming language used in the BT library; and one using the DLR's RMC Advanced Flow Control (RAFCon) [60], as it is a tool to develop concurrent robotic tasks using hierarchical state machine with an intuitive graphical user interface, addressing

[4][Online]. Available: https://en.cppreference.com/w/cpp/thread/barrier

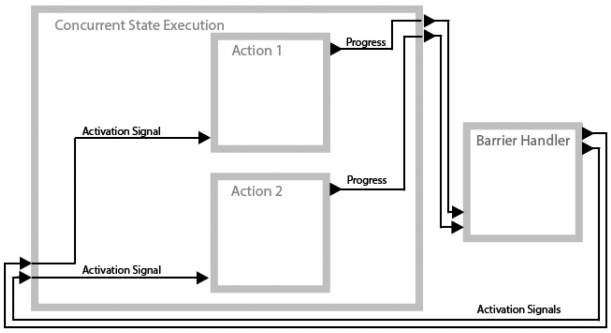

Fig. 17. Concurrent RAFCon state machine for Experiments 3 and 4. *Action 1* and *Action 2* increase the progress if and only the activation signal (their input) equals 1. The *Concurrent State Execution*, which is a RAFCon `concurrency-state` and executes the two substates (*Action 1* and *Action 2*) concurrently. The *Barrier Handler* computes the activation signals for the actions (i.e., it is set to 0 if the action's progress surpasses the current barrier (for absolute synchronization) or the other action's progress by Δ (for relative synchronization); it is set to 1 otherwise).

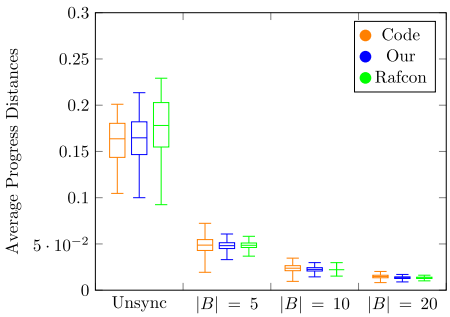

Fig. 18. Boxplot of the progress distances of Experiment 3 with different numbers of barriers $|\mathcal{B}|$ for each method. $|\mathcal{B}| = 0$ corresponds to the unsynchronized execution. We compare the performance obtained using C++ primitives (Code), the proposed approach (Our), and the one Rafcon (Rafcon).

similar issues of BTs.[5] Fig. 17 shows the concurrent state machine developed in Rafcon for the Experiments 3 and 4.

Experiment 3: Consider the BT in Fig. 14(a) where the progress decorator implements an absolute synchronization with equidistant barriers (i.e., a barrier at each $\frac{1}{|B|}$ progress) and the sub-BTs are such that the progress profile of each $\mathcal{T}_i$ holds (6) with $\bar{\omega} = 0.015$.

The model above describes the same BT used in Experiment 1 with the given value for $\bar{\omega}$.

Fig. 18 shows the results of running 10,000 times the BT in Experiment 1 with the different approaches. Note that the unsynchronized setting (e.g., $|\mathcal{B}| = 0$) yields similar values for the different approaches. Hence the boilerplate code of the approach does not affect the performance.

Experiment 4: Consider the BT in Fig. 14(b) where the progress decorator implements a relative synchronization and the sub-BTs are such that the progress profile of each $\mathcal{T}_i$ holds (6) with $\bar{\omega} = 0.015$.

The model above describes the same BT used in Experiment 2 with the given value for $\bar{\omega}$.

[5]Both implementations are [Online]. Available: github.com/miccol/TRO2021-code

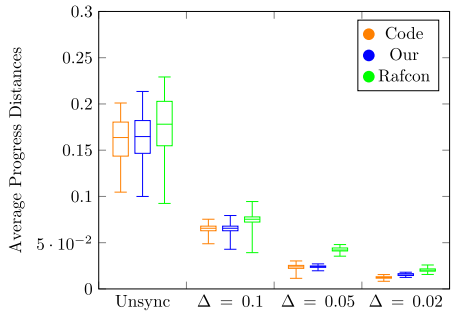

Fig. 19. Boxplot of the progress distances of Experiment 4 with different values of Δ for each method. $\Delta = 1$ is the unsynchronized execution.

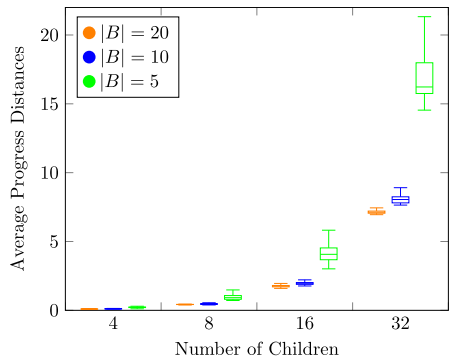

Fig. 20. Boxplot of the predictability distances of Experiment 7 with different values for Δ and $\bar{\omega}$. $\Delta = 1$ corresponds to the unsynchronized execution.

Fig. 19 shows the results of running 10,000 times the BT in Experiment 4 with the different approaches. We make the same observation of the previous experiment. Note that, as in the previous experiment, the unsynchronized setting (e.g., $\Delta = 1$) yields similar values.

Remark 8: Our solution keeps the same order of magnitude as the computer programming ones (the most efficient from a computation point of view) and outperforms the one of Rafcon, while also keeping the advantages of BT over state machines described in the literature [1].

How the number of children to synchronize affects the performance: We now present two experiments that show how the approach scales with the number of children.

Experiment 5: Consider a set of BTs that describe the absolute progress synchronization of a different numbers of actions [Fig. 14(a) shows an example of such BT with two actions]. In each BT, the actions' progress holds (6), with $\alpha = 0.03$ and $\bar{\omega} = 0.015$.

Fig. 20 shows the results of running 10,000 times the BT in Experiment 5 with different numbers of children. We note how the performance decays linearly with the number of children (the number of children increases exponentially in the figure).

Experiment 6: Consider a set of BTs that describe the absolute progress synchronization of different numbers of actions [Fig. 14(a) shows an examples of such BT with two actions]. In each BT, the actions' progress holds (6), with $\alpha = 0.03$ and $\bar{\omega} = 0.015$.

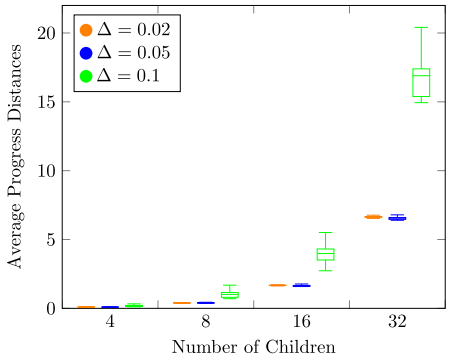

Fig. 21. Boxplot for predictability distance of Experiment 7 with different values for Δ and $\bar{\omega}$. $\Delta = 1$ corresponds to the unsynchronized execution.

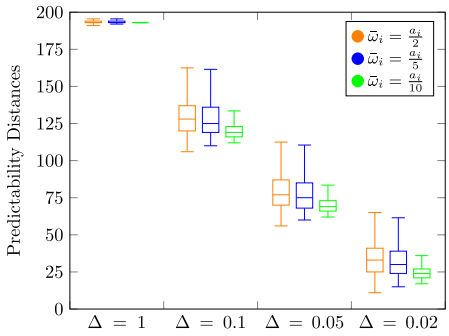

Fig. 22. Boxplot for predictability distance of Experiment 7 with different values for Δ and $\bar{\omega}$. $\Delta = 1$ corresponds to the unsynchronized execution.

Fig. 21 shows the results of running 10,000 times the BT in Experiment 5 with different numbers of children. Similar to the previous experiment. The performance decays linearly with the number of children.

As expected, in both absolute and relative synchronization settings, the number of children deteriorates the progress synchronization performance. This is due to the fact that the children's progresses surpass one another, increasing the progress distance.

How the number of barriers affects the predictability: We now present an experiment that highlights how the number of barriers for an absolute synchronized parallel node affects the predictability of an execution.

Experiment 7: Consider the BT of Fig. 12 where the progress decorator implements a relative synchronization and the action "Arm Movement," whose progress is to be imposed, has a progress defined such that it holds (8) as follows:

$$p_2(x_k) = \begin{cases} 0 & \text{if } k = 0 \\ p_2(x_{k-1}) + 2 + \omega_i(x_k), & \text{otherwise} \end{cases} \tag{8}$$

whereas the progress of the action "Profile" holds (9) as follows:

$$p_1(x_k) = \begin{cases} 0 & \text{if } k = 0 \\ p_1(x_{k-1}) + 0.1, & \text{otherwise.} \end{cases} \tag{9}$$

Fig. 22 reports the results of Experiment 7. We observe worse performance with larger $\bar{\omega}$ and Δ.

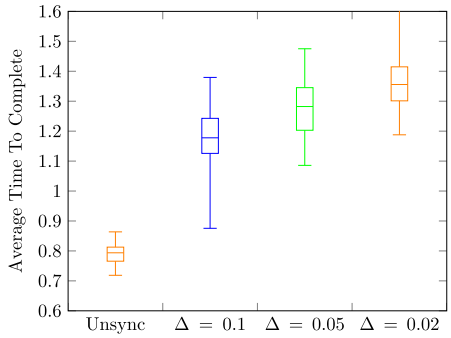

Fig. 23. Boxplot for time to complete.

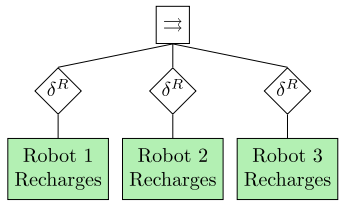

Fig. 24. BT encoding the desired behavior of Experiment 8.

Remark 9: In the experiments above, we showed how a designer could synchronize the progress of several subtrees in a noninvasive fashion. The designer can tune the number of barriers for the absolute synchronization and the threshold value for the relative synchronization. However, as mentioned above, synchronizations between actions may deteriorate other performances. Fig. 23 shows the average times to complete the executions for Experiment 4. Similar results were found for absolute progress synchronization.

How the priority increment function g affects the execution in resource synchronization: We now present two examples of resource synchronization and how the shape of the increment function leads to different behaviors. As mentioned above, this synchronization is done among subtrees that have equal priority in accessing the resources. The function g provides the designer a way to shape their resource allocation strategy. Experiments 8 and 9 show an example of usage of the resource synchronization decorator with different settings for the function g.

Experiment 8 (Greedy dining robots): This experiment is the Dining Philosopher Problem [61] with a twist. Consider three robots that sit in a round table with three cables: Cable A, Cable B, and Cable C. Each cable sits between two robots such that the Robot 1 can grab Cable A and B, Robot 2 can grab Cable B and C, and Robot 3 can grab Cable C and A. Each robot needs two cables to charge its battery. This example represents those cases in which several software components (controlling different robots or different parts on the same robot) need to access a shared resource.

The BT in Fig. 24 encodes a behavior of these robots that ensures resource synchronization. At each tick, the action "Robot i Recharges" increases the battery level by 10% of its full capacity.

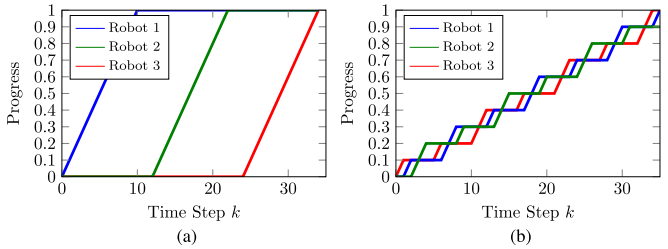

Fig. 25. Progress profiles of the BTs of Experiments 8 and 9. (a) Experiment 8. (b) Experiment 9.

The progress profile follows the battery level as follows:

$$p_i(x_k) = \begin{cases} 0 & \text{if } k = 0 \\ p_i(x_{k-1}) + 0.1, & \text{otherwise} \end{cases} \tag{10}$$

$\mathcal{T}_1$, $\mathcal{T}_2$, and $\mathcal{T}_3$ are such that

$$Q_1(x_k) = \begin{cases} \{A, B\} & \text{if } p(x_k) < 1 \\ \emptyset & \text{otherwise} \end{cases} \tag{11}$$

$$Q_2(x_k) = \begin{cases} \{B, C\} & \text{if } p(x_k) < 1 \\ \emptyset & \text{otherwise} \end{cases} \tag{12}$$

$$Q_3(x_k) = \begin{cases} \{C, A\} & \text{if } p(x_k) < 1 \\ \emptyset & \text{otherwise.} \end{cases} \tag{13}$$

The g function are defined as follows:

$$g_i(x_k) = 0. \tag{14}$$

That is, the priority does not change when the action does not receive ticks.

Fig. 25 shows the progress profile of the two BT. We see how, once a robot acquires the two wires, the wires are assigned to that robot until it no longer requires it (i.e., the battery is fully charged).

Experiment 9 (Fair dining robots): Consider the three robot of the Experiment 8 above, with the difference in the definitions of the g functions

$$g_i(x_k) = 1. \tag{15}$$

That is, the priority increases when the action does not receive ticks.

Fig. 25(b) shows the progress profile of the two BT. We see how the wires are allocated in a "fair" fashion.

The two experiments above show that the choice of the function g becomes crucial to avoid starvation. In Section VIII, we will prove under which circumstances the BT execution avoids starvation. Note that by tuning $g_i(x_k)$, we can achieve different profiles of execution, equivalent to the assignment of a quantum of time received by each robot when they get access to the shared resource.

B. Real World Validation

This section presents the experimental validation implemented on real robots. The literature inspired our experiments.

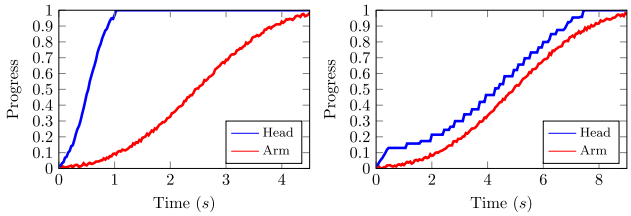

Fig. 26. Progress values for Experiment 10 with (right) and without (left) synchronization.

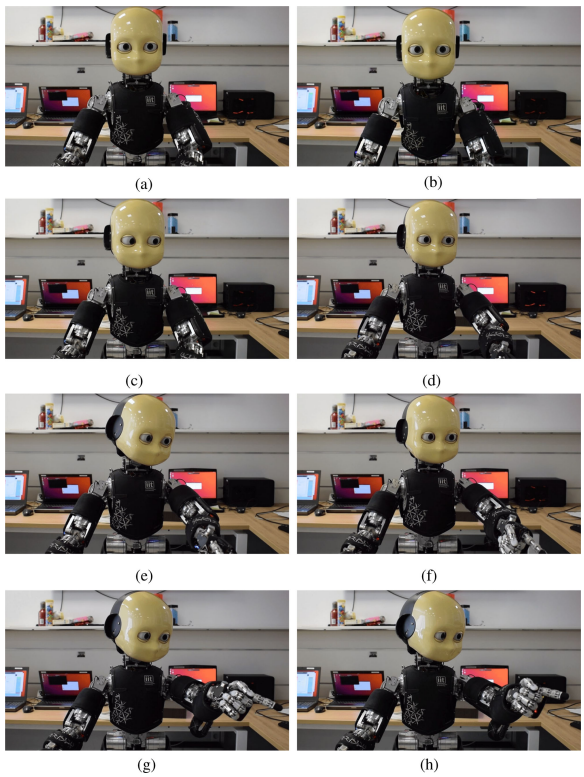

Fig. 27. Execution steps of Experiment 10 with (right) and without (left) synchronization. (a) [Unsync] Initial state. (b) [Sync] Initial state. (c) [Unsync] Robot starts moving the head. (d) [Sync] Robot moves head and arm slowly. (e) [Unsync] Robot finishes to move the head while the arm is still moving. (f) [Sync] Robot keeps moving head and arm slowly. (g) [Unsync] Robot finishes to move the arm. (h) [Sync] Robot finishes to move the head and arm.

Progress synchronization: In Experiment 10, below we present an implementation of Example 2 above, motivated by the impact of contingent behaviors on the quality of verbal human–robot interaction [43].

Experiment 10 (iCub Robot): An iCub robot [62] has to look and point to a given direction. Fig. 26 shows the progress plots for the two actions in the synchronized and unsynchronized case. Fig. 27 shows the progress profiles of the two actions using the iCub Action Rendering Engine[6], where we send concurrently the command to look and point at the same coordinate; and the ones using the BT in Fig. 28 with a relative synchronization and a threshold value of $\Delta = 0.1$, where the actions look and point performs small steps toward the desired coordinate.

[6][Online]. Available: https://robotology.github.io/robotology-documentation/doc/html/group__actionsRenderingEngine.html

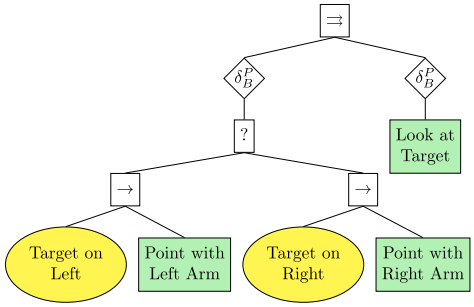

Fig. 28. BT encoding the behavior of Experiment 10.

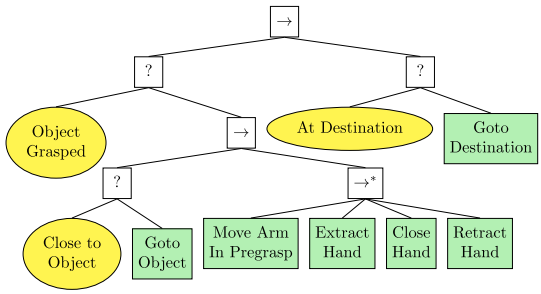

Fig. 29. Original BT encoding the behavior of Experiment 11. The node labeled with $\rightarrow^*$ represents a sequence with memory.

The unsynchronized execution looks unnatural as the head moves way faster than the arm. The synchronization allowed a reduction of the average progress distance from 0.4176 to 0.0964. From the plot in Fig. 26 we note how, with the synchronized execution, the head stops as soon as its progress surpasses the one of the arm by 0.1. Then it moves slower. Moreover, the synchronized execution completes the task in about double the time. This is due to the fact that smaller movements in iCub are performed slowly, and the synchronized execution breaks down the action in small steps.

Resource synchronization: We now present the use case of a resource synchronization mechanism. We took a BT used for a use case for an Integrated Technical Partner European Horizon H2020 project RobMosys[7] and then we used the resource synchronization mechanism to parallelize some tasks executed sequentially using the classical formulation of BTs. As noted in the BT literature [48], [53] turning a sequential behavior execution into a concurrent one becomes much simpler in BTs compared to FSMs.

Experiment 11 (R1 Robot): An R1 robot [63] has to pick up an object from the user's hand and then navigate toward a predefined destination. To grasp the object from the user's hand, the robot has to put the arm in a pregrasp position, extend its hand,[8] grasp the object, and finally, retract the hand.

The BT in Fig. 29 encodes the behavior of the robot designed using the classical BT nodes and Fig. 30 shows some execution steps, as done in the original project above.

However, the robot can execute the pregrasp motion as well as the extraction of the arm while it approaches the user and the retraction of the arm while it navigates toward the destination, whereas the grasping action needs the robot to be still. Hence we can execute the pregrasp and postgrasp action while the robot moves, speeding up the execution. The BT in Fig. 31 models such behavior, taking advantage of the resource synchronization, where the action "Goto" and "Close Hand" allocate the resource *Mobile Base* as long as they are running. Fig. 32 shows

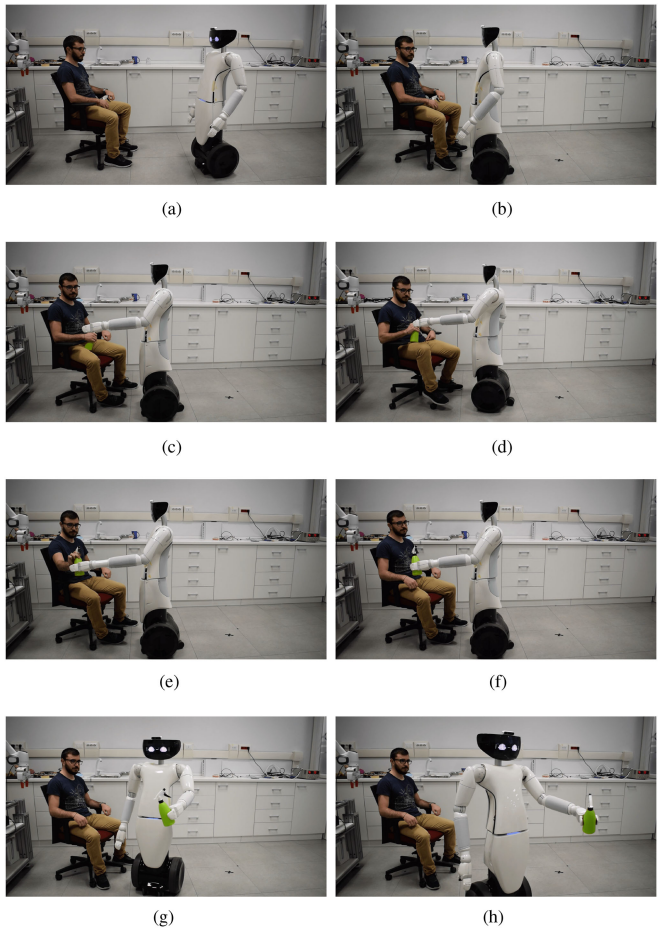

Fig. 30. Execution screenshots of Experiment 11 running the BT in Fig. 29. (a) Robot approaches the user. Action executed: "Goto Object." (b) Robot reaches the user. Action executed: "Goto Object." (c) Robot moves the arm in pregrasp position. Action executed: "Move Arm." (d) Robot extracts the hand. Actions executed: "Extract Hand." (e) Robot closes the hand. Actions executed: "Close Hand." (f) Robot retracts the hand. Actions Executed: "Retract Hand." (g) Robot moves toward the destination. Actions executed: "Goto Destination." (h) Robot moves toward the destination. Actions executed: "Goto Destination."

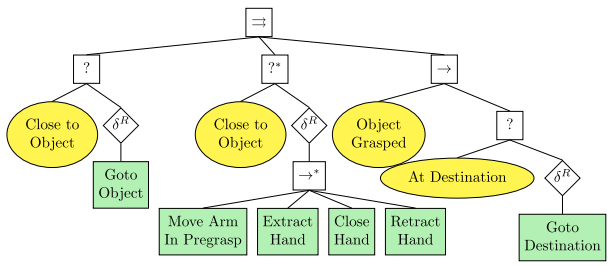

Fig. 31. BT encoding the behavior of Experiment 11 using the resource synchronization mechanism. This BT is significantly simpler than the one in Fig. 29.

[7][Online]. Available: https://scope-robmosys.github.io/

[8]The arm has a prismatic joint in the wrist.

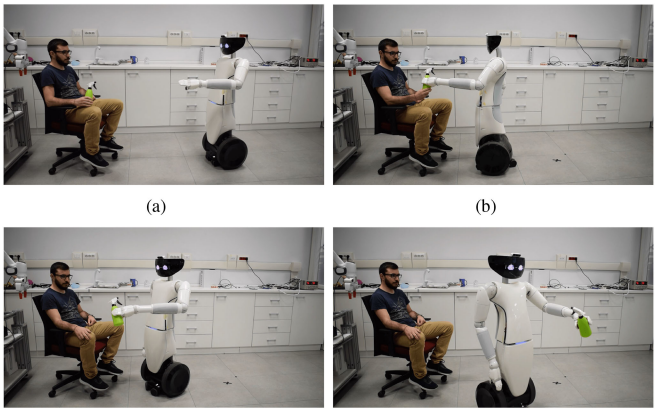

Fig. 32. Execution screenshots of Experiment 11 running the BT in Fig. 31. (a) Robot moves toward the user while positioning the arm and hand. Actions executed: "Goto Object," "Move Arm In Pregrasp," and "Extract Hand." (b) Robot reaches the user and then closes the hand. Action executed: "Close Hand." (c) Robot moves toward the destination while retracting the hand. Actions executed: "Goto Destination" and "Retract Hand." (d) Hand gets retracted. Action executed: "Goto Destination."

some execution steps. The concurrent execution of some actions allows a faster overall behavior as in [48] and [53].

Perpetual Actions: We now present an experiment where we show the applicability of our approach with perpetual actions. Experiment 12 below presents an implementation of Example 3, inspired by the literature [55].

Experiment 12 (Panda robot): An industrial manipulator has to insert a piston into a hollow cylinder. The piston's rod and the piston's head are attached via a revolute joint. To correctly insert the rod, the robot must keep it aligned during the insertion into the cylinder. During the execution, the end-effector gets misaligned, requiring the robot to realign the rod. Fig. 10(b) depicts the BT that encodes this task. The action progress equals the ones of Example 3. Fig. 33 shows the execution steps of this experiment with and without synchronization. We see how the synchronized execution fails since the robot inserts the piston too fast for the alignment subbehavior to have an effect.

VII. Software Library

This section presents the third contribution of the article. We made publicly available an implementation of the nodes presented in this article. The decorators work with the BehaviorTree.CPP engine [64] and the Groot GUI [65]. The user can define the values of barriers $|B|$ (for absolute progress synchronization), the threshold Δ (for relative progress synchronization), or the priority increment function g of Definition 15. The BT can also have independent synchronizations, as shown in the BT of Fig. 34. We made the details available in the library's repository.[9]

[9][Online]. Available: https://github.com/miccol/TRO2021-code

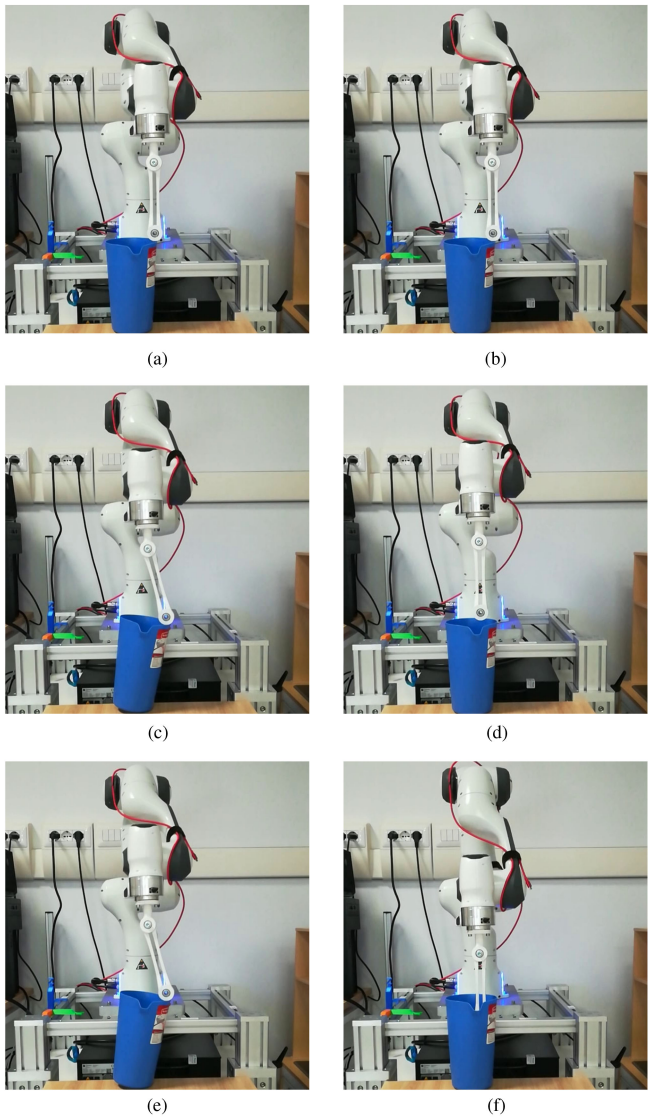

Fig. 33. Execution screenshots of Experiment 12 with (right) and without (left) synchronization. (a) [Unsync] Rod gets misaligned. The robot keeps inserting the rod. (b) [Sync] Rod gets misaligned. The robot stops inserting the rod. (c) [Unsync] Robot aligns the rod while this moves downwards. The rod hits the cylinder. (d) [Sync] Robot aligns the rod. (e) [Unsync] Safety fault stops the execution.

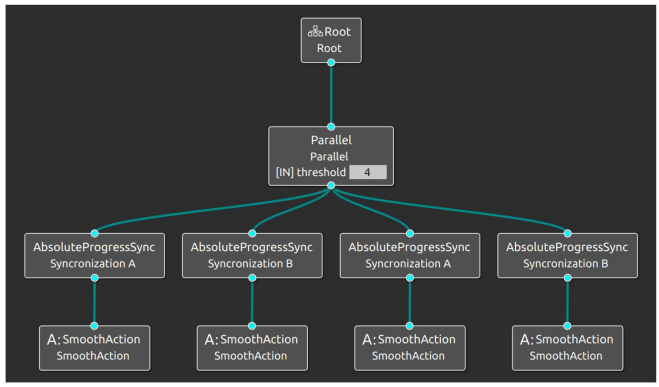

Fig. 34. Concurrent BT of example using the Groot GUI.

A. Implement Concurrent BTs With BehaviorTree.CPP

Listing 1 below shows the code to implement Example 1 above with the BehaviorTree.CPP engine. The user has to instantiate the root and the actions nodes (Lines 1–4), then it defines the barriers (Lines 6–7), it instantiates the decorators (Lines 9–10), and finally, it constructs the BT (Lines 12–16).

Listing 1. Implementation code for the BT in Example 1

```
1  BT::ParallelNode parallel("root",2);
2
3  SyncSmoothAction action1("Arm Movement",0,0.015);
4  SyncSmoothAction action2("Base Movement",0,0.01);
5
6  AbsoluteBarrier barrier({0.1,0.2,0.3,0.4,0.5,0.6,
7                           0.7,0.8,0.9,1.0});
8
9  DecoratorProgressSync dec1("dec1", &barrier);
10 DecoratorProgressSync dec2("dec2", &barrier);
11
12 dec1.addChild(&action1);
13 dec2.addChild(&action2);
14
15 parallel.addChild(&dec1);
16 parallel.addChild(&dec2);
```

B. Implement Concurrent BTs With Groot

We provide a palette of nodes that allow the user to instantiate them using the Groot GUI in a drag-and-drop fashion. The instructions on how to load the palette are available in the library's documentation. Details on how to instantiate and run a generic BT are available in the Groot library's documentation.

VIII. Theoretical Analysis

This section presents the fourth contribution of the article. We first give a set of formal definitions, and then we provide a mathematical analysis of the BT synchronization. As the decorators defined in this article may disable the execution of some subtrees, we need to identify the circumstances that preserve the entire BT's properties.

A. State-Space Formulation of Behavior Trees

The state-space formulation of BTs [1] allows us to study them from a mathematical standpoint. A recursive function call represents the tick. We will use this formulation in the proofs below.

Definition 3 (Behavior tree [1]): A BT is a three-tuple

$$\mathcal{T}_i \triangleq \{f_i, r_i, \Delta t\} \tag{16}$$

where $i \in \mathbb{N}$ is the index of the tree, $f_i : \mathbb{R}^n \to \mathbb{R}^n$ is the right hand side of a difference equation, Δt is a time step and r_i is the return status that can be equal to either *Running*, *Success*, or *Failure*. Finally, let $x_k \triangleq x(t_k)$ be the system state at time t_k, then the execution of a BT $\mathcal{T}_i$ is described by

$$x_{k+1} = f_i(x_k), \tag{17}$$

$$t_{k+1} = t_k + \Delta t. \tag{18}$$

Definition 4 (Sequence compositions of BTs [1]): Two or more BTs can be composed into a more complex BT using a Sequence operator

$$\mathcal{T}_0 = \text{Sequence}(\mathcal{T}_1, \mathcal{T}_2).$$

Then r_0, f_0 are defined as follows:

$$\text{If } x_k \in S_1 \tag{19}$$

$$r_0(x_k) = r_2(x_k) \tag{20}$$

$$f_0(x_k) = f_2(x_k) \tag{21}$$

$$\text{else}$$

$$r_0(x_k) = r_1(x_k) \tag{22}$$

$$f_0(x_k) = f_1(x_k). \tag{23}$$

$\mathcal{T}_1$ and $\mathcal{T}_2$ are called children of $\mathcal{T}_0$.

Remark 10: When executing the new BT, $\mathcal{T}_0$ first keeps executing its first child $\mathcal{T}_1$ as long as it returns Running or Failure.

For notational convenience, we write

$$\text{Sequence}(\mathcal{T}_1, \text{Sequence}(\mathcal{T}_2, \mathcal{T}_3)) = \text{Sequence}(\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3) \tag{24}$$

and similarly for arbitrarily long compositions.

Definition 5 (Fallback compositions of BTs [1]): Two or more BTs can be composed into a more complex BT using a Fallback operator

$$\mathcal{T}_0 = \text{Fallback}(\mathcal{T}_1, \mathcal{T}_2).$$

Then r_0, f_0 are defined as follows:

$$\text{If } x_k \in \mathcal{F}_1 \tag{25}$$

$$r_0(x_k) = r_2(x_k) \tag{26}$$

$$f_0(x_k) = f_2(x_k) \tag{27}$$

$$\text{else}$$

$$r_0(x_k) = r_1(x_k) \tag{28}$$

$$f_0(x_k) = f_1(x_k). \tag{29}$$

For notational convenience, we write

$$\text{Fallback}(\mathcal{T}_1, \text{Fallback}(\mathcal{T}_2, \mathcal{T}_3)) = \text{Fallback}(\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3) \tag{30}$$

and similarly for arbitrarily long compositions.

Definition 6 (Parallel compositions of BTs [1]): Two or more BTs can be composed into a more complex BT using a Parallel operator

$$\mathcal{T}_0 = \text{Parallel}(\mathcal{T}_1, \mathcal{T}_2, M)$$

where $f_0(x) \triangleq (f_1(x), f_2(x))$ and r_0 is defined as follows:

$$\text{If } M = 1$$

$$r_0(x) = \mathcal{S} \text{ If } r_1(x) = \mathcal{S} \vee r_2(x) = \mathcal{S} \tag{31}$$

$$r_0(x) = \mathcal{F} \text{ If } r_1(x) = \mathcal{F} \wedge r_2(x) = \mathcal{F} \tag{32}$$

$$r_0(x) = \mathcal{R} \text{ else} \tag{33}$$

$$\text{If } M = 2$$

$$r_0(x) = \mathcal{S} \text{ If } r_1(x) = \mathcal{S} \wedge r_2(x) = \mathcal{S} \tag{34}$$

$$r_0(x) = \mathcal{F} \text{ If } r_1(x) = \mathcal{F} \vee r_2(x) = \mathcal{F} \tag{35}$$

$$r_0(x) = \mathcal{R} \text{ else.} \tag{36}$$

For notational convenience, we write

$$\text{Parallel}(\mathcal{T}_1, \text{Parallel}(\mathcal{T}_2, \mathcal{T}_3, 2), 2) = \text{Parallel}(\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3, 3) \tag{37}$$

as well as

$$\text{Parallel}(\mathcal{T}_1, \text{Parallel}(\mathcal{T}_2, \mathcal{T}_3, 1), 1) = \text{Parallel}(\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3, 1) \tag{38}$$

and similarly for arbitrarily long compositions.

Definition 7 (Finite time successful [1]): A BT is Finite Time Successful (FTS) with region of attraction R', if for all starting points $x(0) \in R' \subset R$, there is a time τ, and a time $\tau'(x(0))$ such that $\tau'(x) \leq \tau$ for all starting points, and

$x(t) \in R'$ for all $t \in [0, \tau')$ and $x(t) \in S$ for $t = \tau'$.

As noted in the following lemma, exponential stability implies FTS, given the right choices of the sets S, F, R.

Lemma 1 (Exponential stability and FTS [1]): A BT for which x_s is a globally exponentially stable equilibrium of the execution, and $S \supset \{x : ||x - x_s|| \leq \epsilon\}$, $\epsilon > 0$, $F = \emptyset$, $R = \mathbb{R}^n \setminus S$, is FTS.

Safety is the ability to avoid a particular portion of the state-space, which we denote as the *Obstacle Region*. To make statements about the safety of composite BTs, we need the following definition. Details on safe BTs can be found in the literature [1].

Definition 8 (Safeguarding [1]): A BT is safeguarding, with respect to the step length d, the obstacle region $O \subset \mathbb{R}^n$, and the initialization region $I \subset R$, if it is safe, and FTS with region of attraction $R' \supset I$ and a success region S, such that I surrounds S in the following sense:

$$\{x \in X \subset \mathbb{R}^n : \inf_{s \in S} ||x - s|| \leq d\} \subset I \tag{39}$$

where X is the reachable part of the state space $\mathbb{R}^n$.

This implies that the system, under the control of another BT with maximal statespace steplength d, cannot leave S without entering I, and thus, avoiding O [1].

Definition 9 (Safe [1]): A BT is safe, with respect to the obstacle region $O \subset \mathbb{R}^n$, and the initialization region $I \subset R$, if for all starting points $x(0) \in I$, we have that $x(t) \notin O$, for all $t \geq 0$.

B. CBT's Definition

We now formulate additional definitions. We use these definitions to provide a state-space formulation for CBTs (Definition 16 below) and to prove system properties.

Definition 10 (Progress function): The function $p : \mathbb{R}^n \to [0, 1]$ is the progress function. It indicates the progress of the BT's execution at each state.

Definition 11 (Resources): R is a collection of symbols that represents the resources available in the system.

Definition 12 (Allocated resource): Let $\mathcal{N}$ be the set of all the nodes of a BT, the function $\alpha : \mathbb{R}^n \times R \to \mathcal{N}$ is the resource allocation function. It indicates the BT using a resource.

Definition 13 (Resource function): The function $Q : \mathbb{R}^n \to 2^R$ is the resource function. It indicates the set of resources needed for a BT's execution at each state.

Definition 14 (Node priority): The function $\rho : \mathbb{R}^n \to \mathbb{R}$ is the priority function. It indicates the node's priority to access a resource.

Definition 15 (Priority increment function): The function $g : \mathbb{R}^{\mathbb{m}} \to \mathbb{R}$ is the priority increment function. It indicates how the priority changes while a node is waiting for a resource.

We can now define a CBT as BT with information regarding its progress and the resources needed as follows:

Definition 16 (Concurrent BTs): A CBT is a tuple

$$\mathcal{T}_i \triangleq \{f_i, r_i, \Delta t, p_i, q_i\} \tag{40}$$

where i, f_i, Δt, r_i are defined as in Definition 3, p_i is a progress function, and q is a resource function.

A CBT has the functions p_i and q_i in addition to the others of Definition 3. These functions are user-defined for Actions and Condition. For the classical operators, the functions are defined below.

Definition 17 (Sequence compositions of CBTs): Two CBTs can be composed into a more complex CBT using a Sequence operator

$$\mathcal{T}_0 = \text{Sequence}(\mathcal{T}_1, \mathcal{T}_2).$$

The functions r_0, f_0 match those introduced in Definition 4, while the functions p_0, q_0 are defined as follows:

$$\text{If } x_k \in S_1 \tag{41}$$

$$p_0(x_k) = \frac{p_1(x_k) + p_2(x_k)}{2} \tag{42}$$

$$q_0(x_k) = q_2(x_k) \tag{43}$$

else

$$p_0(x_k) = \frac{p_1(x_k)}{2} \tag{44}$$

$$q_0(x_k) = q_1(x_k). \tag{45}$$

Definition 18 (Fallback compositions of CBTs): Two CBTs can be composed into a more complex CBT using a Fallback operator

$$\mathcal{T}_0 = \text{Fallback}(\mathcal{T}_1, \mathcal{T}_2).$$

The functions r_0, f_0 are defined as in Definition 5, while the functions p_0, q_0 are defined as follows:

$$\text{If } x_k \in F_1 \tag{46}$$

$$p_0(x_k) = p_2(x_k) \tag{47}$$

$$q_0(x_k) = q_2(x_k) \tag{48}$$

else

$$p_0(x_k) = p_1(x_k) \tag{49}$$

$$q_0(x_k) = q_1(x_k). \tag{50}$$

Definition 19 (Parallel compositions of CBTs): Two CBTs can be composed into a more complex CBT using a Parallel

operator

$$\mathcal{T}_0 = \text{Parallel}(\mathcal{T}_1, \mathcal{T}_2).$$

The functions r_0, f_0 are defined as in Definition 6, while the functions p_0 and q_0 are defined as follows:

$$p_0(x_k) = \min(p_1(x_k), p_2(x_k)) \tag{51}$$

$$q_0(x_k) = q_1(x_k) \cup q_2(x_k). \tag{52}$$

Remark 11: Conditions nodes do not perform any action. Hence their progress function can be defined as $p(x_k) = 0$ and their resource function as $q(x_k) = \emptyset \; \forall x_k \in \mathbb{R}^n$.

Definition 20 (Absolute barrier): An absolute barrier is defined as

$$b(x_k) \triangleq min\{b_i \in B : \forall T_j \in T, p_j(x_k) \geq b_{i-1} \wedge \\ \wedge \exists \mathcal{T}_k : p_k(x_k) \geq b_i\} \tag{53}$$

with B a finite set of progress values.

Definition 21 (Relative barrier): A relative barrier is defined as

$$b(t_k) \triangleq min_{\mathcal{T}_i \in T}\{p_i\} + \Delta \tag{54}$$

with $\Delta \in [0, 1]$

Definition 22 (Functional formulation of a progress decorator node): A CBT $\mathcal{T}_1$ can be composed into a more complex BT using an Absolute Progress Decorator operator

$$\mathcal{T}_0 = \text{AbsoluteProgress}(\mathcal{T}_1, b(x_k)).$$

Then r_0, f_0, p_0, Q_0 are defined as follows:

$$p_0(x_k) = p_1(x_k) \tag{55}$$

$$Q_0(x_k) = Q_1(x_k) \tag{56}$$

$$\text{If } p_1(x_k) < b(x_k)$$

$$f_0(x_k) = f_1(x_k) \tag{57}$$

$$r_0(x_k) = r_1(x_k) \tag{58}$$

$$\text{else}$$

$$f_0(x_k) = x_k \tag{59}$$

$$r_0(x_k) = \mathcal{R}. \tag{60}$$

With $b(x_k)$ as in Definition 20 for an absolute synchronization or as in Definition 21 for a relative synchronization.

Definition 23 (Functional formulation of a resource decorator node): A Smooth BT $\mathcal{T}_1$ can be composed into a more complex BT using a Resource Decorator operator

$$\mathcal{T}_0 = \text{ResourceDecorator}(\mathcal{T}_1, g).$$

With g as in Definition 15. then r_0, f_0, p_0, Q_0 are defined as follows:

$$p_0(x_k) = p_1(x_k) \tag{61}$$

$$\text{If } (\forall q \in Q_1(x_k) : \alpha(x_k, q) = \mathcal{T}_1 \wedge \rho_1(x_k) \geq \rho_{max}) \\ \vee \alpha(x_k, q) = \emptyset$$

$$r_0(x_k) = r_1(x_k) \tag{62}$$

$$f_0(x_k) = f_1(x_k) \tag{63}$$

$$Q_0(x_k) = Q_1(x_k) \tag{64}$$

$$\alpha(x_k, q) = \begin{cases} \mathcal{T}_1 & \text{if } q \in Q_1(x_k) \\ \emptyset & \text{if } q \notin Q_1(x_k) : \\ & \quad \alpha(x_{k-1}, q) = \mathcal{T}_1 \\ \alpha(x_{k-1}, q) & \text{otherwise} \\ & \quad \rho(x_k) = \rho(x_{k-1}) \end{cases} \tag{65}$$

else

$$r_0(x_k) = \mathcal{R} \tag{66}$$

$$f_0(x_k) = x_k \tag{67}$$

$$Q_0(x_k) = \emptyset \tag{68}$$

$$\rho_1(x_k) = \rho_1(x_{k-1}) + g_1(x_k) \tag{69}$$

with α from Definition 12.

Definition 24 (Active node): A BT node $\mathcal{T}_i$ is said to be active in a given BT if $\mathcal{T}_i$ is either the root node or whenever $r(x_k) = R$, $\mathcal{T}_i$ will eventually receive a tick.

An *active* node is a node that eventually will receive a tick when its returns status is running.

C. Lemmas

The synchronization mechanism proposed in this article may jeopardize the FTS property (described in Definition 7) of a BT. In particular, a FTS BT may no longer receive ticks from decorators proposed in this article as this will wait for another action indefinitely. This relates to the problem of *starvation*, where a process waits for a critical resource and other processes, with a higher priority, prevent access to such resource [61].

Lemma 2 (ProgressSync FTS BTs): Let $\mathcal{T}_1$ and $\mathcal{T}_2$ be two FTS, with region of attraction R_1 and R_2 respectively, and active sub-BTs in the BT $\mathcal{T}$. The sub-BTs $\tilde{\mathcal{T}_1} = DecoratorSync(\mathcal{T}_1, b(x_k))$ and $\tilde{\mathcal{T}_2} = DecoratorSync(\mathcal{T}_2, b(x_k))$ in the BT $\tilde{\mathcal{T}}$ obtained by replacing in $\mathcal{T}$ $\mathcal{T}_1$ and $\mathcal{T}_2$ with $\tilde{\mathcal{T}_1}$ and $\tilde{\mathcal{T}_2}$, respectively, are FTS.

Proof: Since the $\mathcal{T}_1$ and $\mathcal{T}_2$ are active they will receive ticks as long as their return status is running, from (16) and (58), $\tilde{\mathcal{T}_1}$ and $\tilde{\mathcal{T}_2}$ have the same return statuses of $\mathcal{T}_1$ and $\mathcal{T}_2$, respectively, they both will receive ticks as long as they return status is running. Since $\mathcal{T}_1$ and $\mathcal{T}_2$ are FTS with region of attraction R they will eventually reach a state $x_{\bar{k}} \in S_i$, which implies $r_i(x_{\bar{k}}) = S$ hence eventually $p_i(x_{\bar{k}}) = 1$. In such case the the decorator propagate every tick that it receive.

Corollary 1 (of Lemma 2): Let $\mathcal{T}_1$, $\mathcal{T}_2$, $\cdots$, and $\mathcal{T}_N$ be N FTS with region of attraction R_i and active sub-BTs. Each sub-bt $\tilde{\mathcal{T}_i} = DecoratorSync(\mathcal{T}_i, B)$ is FTS if $r_i(x_k) = S \Rightarrow p_i(x_k) = 1$ hold.

Proof: The proof is similar to the one of Lemma 2.

Lemma 3: Let $\mathcal{T}_1$ be a safeguarding BT with respect to the step length d, the obstacle region $O \subset \mathbb{R}^n$, and the initialization region $I \subset R$. Then $\mathcal{T}_0 = ProgressSync(\mathcal{T}, b(x_k))$ is also safeguarding with respect to the step length d, the obstacle region

$O \subset \mathbb{R}^n$, and the initialization region $I \subset R$, for any value of $b(x_k)$.

Proof: From Definition 8, $\mathcal{T}$ holds the following: $\{x : \inf_{s \in S_1} ||x - s|| \leq d\} \subset I$ hence $|f_1(x_k) - x_k| \leq d$ holds. From Definition 10, $f_0(x_k)$ is either $f_1(x_k)$ or x_k, hence $|f_0(x_k) - x_k| \leq d$ holds.

Lemma 4: Let $\mathcal{T}_0 = ResourceDecorator(\mathcal{T}_1, g)$, if $g(x_k) > 0 \; \forall x_k \in \mathbb{R}$ then the execution of $\mathcal{T}_0$ is starvation-free regardless the resources allocated.

Proof: According to (69), since $g(x_k) > 0$, whenever the $\mathcal{T}_0$ does not propagate ticks to the BT $\mathcal{T}_1$ it gradually increases the priority of $\mathcal{T}_1$.

Setting $g(x_k) > 0$ implements aging, a technique to avoid starvation [61]. We could also shape the function g such that it implements different scheduling policies. However, that falls beyond the scope of the article.

IX. Conclusion

This article proposed two new BTs control flow nodes for resource and progress synchronization with different synchronization policies, absolute and relative. We proposed measures to assess the synchronization between different sub-BTs and the predictability of robot execution. Moreover, we observed how design choices for synchronization might affect the performance. The experimental validation supports such observations.

We showed our approach's applicability in a simulation system that allowed us to run the experiments several times in different settings to collect statistically significant data. We also showed the applicability of our approach in real robot scenarios taken from the literature. We provided the source code of our experimental validation and the code for the control flow nodes aforementioned. Finally, we studied the proposed node from a theoretical standpoint, which allowed us to identify the assumptions under which the synchronization does not jeopardize some BT properties.

Acknowledgment

The authors would like to thank, in alphabetical order, F. Bottarel, M. Monforte, L. Nobile, N. Piga, and E. Rampone for the support in the experimental validation.

References

[1] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction (Chapman and Hall/CRC Artificial Intelligence and Robotics Series)*. New York, NY, USA: Taylor & Francis, 2018.

[2] F. Rovida, B. Grossmann, and V. Krüger, "Extended behavior trees for quick definition of flexible robotic tasks," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 6793–6800.

[3] D. Zhang and B. Hannaford, "IkBT: Solving symbolic inverse kinematics with behavior tree," *J. Artif. Intell. Res.*, vol. 65, pp. 457–486, 2019.

[4] A. Csiszar, M. Hoppe, S. A. Khader, and A. Verl, "Behavior trees for task-level programming of industrial robots," in *Proc. Tagungsband des 2. Kongresses Montage Handhabung Industrieroboter*, 2017, pp. 175–186.

[5] E. Coronado, F. Mastrogiovanni, and G. Venture, "Development of intelligent behaviors for social robots via user-friendly and modular programming tools," in *Proc. IEEE Workshop Adv. Robot. Social Impacts*, 2018, pp. 62–68.

[6] C. Paxton, A. Hundt, F. Jonathan, K. Guerin, and G. D. Hager, "CoSTAR: Instructing collaborative robots with behavior trees and vision," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2017, pp. 564–571.

[7] D. Shepherd, P. Francis, D. Weintrop, D. Franklin, B. Li, and A. Afzal, "[engineering paper] an IDE for easy programming of simple robotics tasks," in *Proc. IEEE 18th Int. Work. Conf. Source Code Anal. Manipulation*, 2018, pp. 209–214.

[8] X. Neufeld, S. Mostaghim, and S. Brand, "A hybrid approach to planning and execution in dynamic environments through hierarchical task networks and behavior trees," in *Proc. 14th Artif. Intell. Interactive Digit. Entertainment Conf.*, 2018.

[9] M. Kim *et al.*, "An architecture for person-following using active target search," 2018, *arXiv: 1809.08793*.

[10] N. Axelsson and G. Skantze, "Modelling adaptive presentations in human-robot interaction using behaviour trees," in *Proc. 20th Annu. SIGdial Meeting Discourse Dialogue*, 2019, pp. 345–352.

[11] A. Ghadirzadeh, X. Chen, W. Yin, Z. Yi, M. BjÖrkman, and D. Kragic, "Human-centered collaborative robots with deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 566–571, Apr. 2011.

[12] C. I. Sprague and P. Ögren, "Adding neural network controllers to behavior trees without destroying performance guarantees," 2018, *arXiv: 1809.10283*.

[13] B. Banerjee, "Autonomous acquisition of behavior trees for robot control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 3460–3467.

[14] B. Hannaford, "Hidden Markov models derived from behavior trees," 2019, *arXiv: 1907.10029*.

[15] E. Scheide, G. Best, and G. A. Hollinger, "Learning behavior trees for robotic task planning by Monte Carlo search over a formal grammar," in *Proc. RSS Workshop Learn. Task Motion Plan.*, 2020.

[16] E. Safronov, M. Vilzmann, D. Tsetserukou, and K. Kondak, "Asynchronous behavior trees with memory aimed at aerial vehicles with redundancy in flight controller," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 3113–3118.

[17] C. I. Sprague, Ö. Özkahraman, A. Munafo, R. Marlow, A. Phillips, and P. Ögren, "Improving the modularity of AUV control systems using behaviour trees," 2018, *arXiv: 1811.00426*.

[18] P. Ögren, "Increasing modularity of UAV control systems using computer game behavior trees," in *Proc. AIAA Guid., Navigation Control Conf.*, 2012.

[19] T. S. Bruggemann *et al.*, "Analysing the reliability of multi UAV operations," in *Proc. 17th Aust. Int. Aerosp. Congress*, 2017, pp. 406–411.

[20] D. Crofts, T. S. Bruggemann, and J. J. Ford, "A behaviour tree-based robust decision framework for enhanced UAV autonomy," in *Proc. 17th Australian Int. Aerosp. Congr.*2017, pp. 412–417.

[21] M. Molina, A. Carrera, A. Camporredondo, H. Bavle, A. Rodriguez-Ramos, and P. Campoy, "Building the executive system of autonomous aerial robots using the aerostack open-source framework," *Int. J. Adv. Robot. Syst.*, vol. 17, no. 3, 2020, Art. no. 1729881420925000.

[22] O. Biggar and M. Zamani, "A framework for formal verification of behavior trees with linear temporal logic," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 2341–2348, Apr. 2020.

[23] T. G. Tadewos, L. Shamgah, and A. Karimoddini, "On-the-fly decentralized tasking of autonomous vehicles," in *Proc. IEEE 58th Conf. Decis. Control*, 2019, pp. 2770–2775.

[24] J. Kuckling, A. Ligot, D. Bozhinoski, and M. Birattari, "Behavior trees as a control architecture in the automatic modular design of robot swarms," in *Proc. Int. Conf. Swarm Intell.*, 2018, pp. 30–43.

[25] Ö. Özkahraman and P. Ögren, "Combining control barrier functions and behavior trees for multi-agent underwater coverage missions," in *Proc. 59th IEEE Conf. Decis. Control*, 2020, pp. 5275–5282.

[26] O. Biggar, M. Zamani, and I. Shames, "A principled analysis of behavior trees and their generalisations," 2020, *arXiv: 2008.11906*.

[27] P. de la Cruz, J. Piater, and M. Saveriano, "Reconfigurable behavior trees: Towards an executive framework meeting high-level decision making and control layer features," in *Proc. IEEE Int. Conf. Systems, Man, Cybern.*, 2020, pp. 1915–1922.

[28] P. Ögren, "Convergence analysis of hybrid control systems in the form of backward chained behavior trees," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6073–6080, Oct. 2020.

[29] "BostonDynamics spot SDK," 2020. [Online]. Available: https://www.bostondynamics.com/spot2_0

[30] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 2718–2725.

[31] F. Martín, J. Ginés, F. J. Rodríguez, and V. Matellán, "PlanSys2: A planning system framework for ROS2," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2021.
[32] D. Isla, "Handling complexity in the halo 2 AI," in *Proc. Game Developers Conf.*, 2005.
[33] I. Millington and J. Funge, *Artificial Intelligence for Games*. Boca Raton, FL, USA: CRC Press, 2009.
[34] J. Lunze and F. Lamnabhi-Lagarrigue, *Handbook of Hybrid Systems Control: Theory, Tools, Applications*. Cambridge, U.K.: Cambridge Univ. Press, 2009.
[35] C. Sloan, J. D. Kelleher, and B. Mac Namee, "Feasibility study of utility-directed behaviour for computer game agents," in *Proc. 8th Int. Conf. Adv. Comput. Entertainment Technol.*, 2011, pp. 1–6.
[36] A. J. Champandard, "10 reasons the age of finite state machines is over," AIGameDev.com, 2007. [Online]. Available: http://aigamedev.com/open/article/fsm-age-is-over
[37] E. W. Dijkstra, "Go to statement considered harmful [*letter to the editor*]," *Commun. ACM*, vol. 11, no. 3, pp. 147–148, 1968.
[38] F. Rubin, "Goto considered harmful considered harmful," *Commun. ACM*, vol. 30, no. 3, pp. 195–196, 1987.
[39] B. A. Benander, N. Gorla, and A. C. Benander, "An empirical study of the use of the goto statement," *J. Syst. Softw.*, vol. 11, no. 3, pp. 217–223, 1990.
[40] R. L.Ashenhurst, "ACM forum," *Commun. ACM*, vol. 30, no. 5, pp. 350–355, May 1987. [Online]. Available: https://doi.org/10.1145/22899.315729
[41] M. Iovino, E. Scukins, J. Styrud, P. Ögren, and C. Smith, "A survey of behavior trees in robotics and AI," 2020, *arXiv: 2005.05842*.
[42] G. Taubenfeld, *Synchronization Algorithms and Concurrent Programming*. London, U.K.: Pearson Education, 2006.
[43] K. Fischer, K. Lohan, J. Saunders, C. Nehaniv, B. Wrede, and K. Rohlfing, "The impact of the contingency of robot feedback on HRI," in *Proc. IEEE Int. Conf. Collaboration Technol. Syst.*, 2013, pp. 210–217.
[44] J. Lee, J. F. Kiser, A. F. Bobick, and A. L. Thomaz, "Vision-based contingency detection," in *Proc. 6th Int. Conf. Hum.-Robot Interact.*, 2011, pp. 297–304.
[45] S. Kopp *et al.*, "Towards a common framework for multimodal generation: The behavior markup language," in *Proc. Int. workshop Intell. Virtual Agents*, 2006, pp. 205–217.
[46] M. Colledanchise and L. Natale, "Improving the parallel execution of behavior trees," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 7103–7110.
[47] M. Colledanchise and L. Natale, "Analysis and exploitation of synchronized parallel executions in behavior trees," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 6399–6406.
[48] S. G. Brunner, A. DÖmel, P. Lehner, M. Beetz, and F. Stulp, "Autonomous parallelization of resource-aware robotic task nodes," *IEEE Robot. Autom. Lett.*, vol. 4, no. 3, pp. 2599–2606, Jul. 2019.
[49] A. Champandard, "Enabling concurrency in your behavior hierarchy," *AIGameDev. com*, 2007.
[50] B. G. Weber, P. Mawhorter, M. Mateas, and A. Jhala, "Reactive planning idioms for multi-scale game AI," in *Proc. IEEE Comput. Intell. Games Symp.*, 2010, pp. 115–122.
[51] M. Mateas and A. Stern, "A behavior language for story-based believable agents," *IEEE Intell. Syst.*, vol. 17, no. 4, pp. 39–47, Jul./Aug. 2002.
[52] R. A. Agis, S. Gottifredi, and A. J. García, "An event-driven behavior trees extension to facilitate non-player multi-agent coordination in video games," *Expert Syst. Appl.*, vol. 155, 2020, Art. no. 113457.
[53] M. Colledanchise, A. Marzinotto, D. V. Dimarogonas, and P. Ögren, "The advantages of using behavior trees in multi-robot systems," in *Proc. 47st Int. Symp. Robot. VDE*, 2016, pp. 1–8.
[54] Q. Yang and R. Parasuraman, "Hierarchical needs based self-adaptive framework for cooperative multi-robot system," in *Proc. IEEE Int. Conf. Systems, Man, Cybern.*, 2020, pp. 2991–2998.
[55] F. Rovida, D. Wuthier, B. Grossmann, M. Fumagalli, and V. Krüger, "Motion generators combined with behavior trees: A novel approach to skill modelling," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 5964–5971.
[56] M. Colledanchise and L. Natale, "On the implementation of behavior trees in robotics," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5929–5936, Jul. 2021.
[57] S. Chitta, B. Cohen, and M. Likhachev, "Planning for autonomous door opening with a mobile manipulator," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2010, pp. 1799–1806.
[58] A. Hern, "Boston dynamics crosses new threshold with door-opening dog," *Guardian*, 2018. [Online]. Available: https://www.theguardian.com/technology/2018/feb/13/boston-dynamics-spot-mini-door-opening-dog-crosses-threshold
[59] J. Yao, Q. Huang, and W. Wang, "Adaptive CGFS based on grammatical evolution," *Math. Problems Eng.*, vol. 2015, 2015, Art. no. 197306.
[60] S. G. Brunner, F. Steinmetz, R. Belder, and A. DÖmel, "RAFCON: A graphical tool for engineering complex, robotic tasks," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2016, pp. 3283–3290.
[61] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*. London, U.K.: Pearson, 2015.
[62] G. Metta, G. Sandini, D. Vernon, L. Natale, and F. Nori, "The ICUB humanoid robot: An open platform for research in embodied cognition," in *Proc. 8th Workshop Perform. Metrics Intell. Syst.*, 2008, pp. 50–56.
[63] A. Parmiggiani *et al.*, "The design and validation of the R1 personal humanoid," in *Proc. IEEE/RSJ Int. Conf. Intell. robots Syst.*, 2017, pp. 674–680.
[64] "BehaviorTree.CPP, Behavior trees library in c++ batteries included," 2020. [Online]. Available: https://github.com/BehaviorTree/BehaviorTree.CPP
[65] "Groot. Graphical editor to create behaviorTrees. compliant with behaviortree.CPP," 2020. [Online]. Available: https://github.com/BehaviorTree/Groot

Michele Colledanchise (Member, IEEE) received the B.S. and M.S. degrees in automatic control from the Polytechnic University of Marche, Ancona, Italy, in 2010 and 2012, respectively, and the Ph.D. degree in computer science from the Kungliga Tekniska Högskolan (KTH), Stockholm, Sweden, in 2017, supervised by Prof. Petter Ögren.

He is currently a Postdoctoral Researcher with the Center for Robotics and Intelligent Systems (CRIS) with the Italian Institute of Technology (IIT), Genoa, Italy, working in Dr. Lorenzo Natale's group. Previously, he was a Postdoctoral Researcher with KTH, working with Prof. Petter Ögren. In the Spring semester 2016, he visited the Control and Dynamical Systems lab, Caltech, Pasadena, CA, USA, supervised by Prof. Richard Murray. His research interests include behavior trees, task planning under uncertainty, and robot behavior verification.

Lorenzo Natale (Senior Member, IEEE) received the electronic engineering degree (with honours) and the Ph.D. degree in robotics from the University of Genoa, Genoa, Italy, in 2000 and 2004, respectively.

He is Tenured Senior Researcher with the Italian Institute of Technology, Genoa, Italy. He was later Postdoctoral Researcher with the MIT Computer Science and Artificial Intelligence Laboratory, Cambridge, MA, USA. He is a Visiting Professor with the University of Manchester, England, U.K. His research interests include vision, tactile sensing for grasping and manipulation, and software architectures for robotics.

Dr. Natale serves as the Specialty Chief Editor for the Humanoid Robotics Section of *Frontiers in Robotics and AI*, Associate Editor for IEEE TRANSACTIONS ON ROBOTICS. He is Ellis Fellow and a Core Faculty of the Ellis Genoa Unit.